



Final Report

DeHalu: Agentic Hallucination Detection and Mitigation in Local CodeLLMs

SE801: Software Project Lab III

Submitted by

Farhan Tausif
BSSE1440

Supervised by

Dr. Kazi Muheymin-Us-Sakib
Professor
Institute of Information Technology
University of Dhaka

Submission Date: 11–10-2026

Supervisor's Signature

Table of Contents

1. Project Title	1
1.1 Abstract	1
1.2 Project Overview	1
1.2.1 Rationale and Objectives	1
1.2.2 Scope and Evolution	3
1.2.3 Deliverables and Report Organization	4
1.2.4 Evidence Basis and Terminology	4
2. Requirements Analysis	5
2.1 Functional Requirements	6
2.1.1 Prompt Intake and Inference	6
2.1.2 Code Generation	6
2.1.3 Claim Extraction	6
2.1.4 Execution-Free Static Analysis	7
2.1.5 Tool-Backed Package and API Validation	7
2.1.6 Quantitative Metric Engine	7
2.1.7 LLM-as-a-Judge Pool Verification	8
2.1.8 Chain-of-Verification	8
2.1.9 Policy Decision and Mitigation	8
2.1.10 Evidence Reporting and Monitoring	8
2.2 Non-Functional Requirements	9
3. Component Level Design	10
3.1 Overall System Design	10
3.2 Component-wise High-Level Design	11
3.2.1 Prompt Composer and Workspace Controller	11
3.2.2 API Boundary and Domain Contracts	12
3.2.3 Durable Job Worker and Run Orchestrator	12
3.2.4 Local Generator and Artifact Normalizer	12
3.2.5 Claim Extraction and Language Adapters	13
3.2.6 Static Parser and SAST Executor	13
3.2.7 Symbols, Packages and API Catalogs	13
3.2.8 Metric Engine	14
3.2.9 Judge Pool and Consensus	14
3.2.10 Chain-of-Verification	14
3.2.11 Policy Gateway and Repair Controller	15
3.2.12 Persistence and Evidence Repository	15
3.3 Domain and Design Classes	16
3.4 Metrics and Decision Rules	17
3.5 Behavioral Design	19
3.5.1 Submission, Queue and Replay	19

3.5.2 Run Lifecycle and Failure Preservation	19
3.5.3 Repair and Evidence Provenance	20
3.6 API and Event Contracts	20
3.7 Deployment Design	22
4. Interface Design	22
4.1 Users, Tasks and Navigation	23
4.2 Interface Objects and Actions	23
4.3 Events and State Transitions	25
4.4 Interface Screens and Evidence Provenance	26
4.4.1 Initial Prompt Workspace	26
4.4.2 Advanced Context	27
4.4.3 Code and Warning Outcome	27
4.4.4 Evidence Overview	28
4.4.5 Metrics and Missing Measurements	28
4.4.6 Provider Failure and Judge Coverage	29
4.4.7 Repair Comparison	29
4.4.8 Clarification and Rejection	30
4.5 Responsive and Accessibility Considerations	31
5. Testing	31
5.1 Testing Approach and Strategy	32
5.2 Item Pass/Fail Criteria	32
5.3 Observed Results	33
5.4 Detailed Backend Test Cases	35
5.4.1 TC-B01: Language inference and ambiguity	35
5.4.2 TC-B02: Eight-language parser and extraction matrix	35
5.4.3 TC-B03: Lexical scopes, aliases and invalid APIs	35
5.4.4 TC-B04: Registry absence versus outage	35
5.4.5 TC-B05: Static unsafe-pattern and repetition evidence	36
5.4.6 TC-B06: Metric count and missing entropy semantics	36
5.4.7 TC-B07: CoVe contradiction precedence and missing judges	36
5.4.8 TC-B08: Acceptance requires complete evidence	36
5.4.9 TC-B09: Generation artifact validation	37
5.4.10 TC-I01: Exclusive queue claim and persisted evidence links	37
5.4.11 TC-I02: Cancellation and expired worker lease	37
5.4.12 TC-I03: Clarification resumes the same run	37
5.4.13 TC-I04: Repair budget and repeated verification	37
5.4.14 TC-I05: HTTP enqueue, replay, cancel and export	38
5.4.15 TC-I06: Failure retains partial output and stage status	38
5.4.16 TC-I07: Repeated artifact stops repair without acceptance	38
5.4.17 TC-I08: Separate worker entry point	38
5.4.18 TC-I09: Incomplete metadata normalization preserves source	38

5.4.19 TC-I10: Repair cannot hide an unverified new dependency	39
5.4.20 TC-I11: API pipeline persistence and compatibility stream	39
5.5 Frontend and Browser Test Cases	39
5.5.1 TC-F01: Stream Parsing and Contract Handling	39
5.5.2 TC-F02: Durable Workspace Restoration and Stale Responses	39
5.5.3 TC-F03: Historical Attempt Selection and New Draft	40
5.5.4 TC-F04: Evidence Navigation and Unavailable Data	40
5.5.5 TC-B01: Clean, Repaired, Rejected and Degraded Completion	40
5.5.6 TC-B02: Responsive Layout and Source Navigation	40
5.6 Defect Analysis and Evaluation Limitations	40
5.7 Risks and Contingencies	41
5.8 Reproduction and Evidence Location	42
6. User Manual	42
6.1 System Requirements and Prerequisites	42
6.2 Installation and Configuration	43
6.2.1 Obtain the Source	43
6.2.2 Select Generator and Review Mode	44
6.2.3 Start the Database and Install Backend Dependencies	44
6.2.4 Start API, Worker and Frontend	45
6.3 Readiness and First Use	45
6.4 Main User Workflows	45
6.4.1 Compose a Task and Supply Context	45
6.4.2 Answer Clarification	46
6.4.3 Monitor Generation and Checks	46
6.4.4 Interpret the Verification Decision	46
6.4.5 Inspect Claims, Findings and Source Links	46
6.4.6 Interpret Metrics and Judges	47
6.4.7 Compare Attempts and Copy Code	47
6.5 History and Evidence Export	47
6.6 Troubleshooting and Recovery	47
6.7 Validation for Maintainers	49
7. Conclusion	49
7.1 Achieved Outcomes	49
7.2 Limitations	50
7.3 Future Work	50
References	51

List of Figures

Figure-1: Overall system architecture and verification boundary	12
Figure-2: Verification stages and bounded repair workflow	13
Figure-3: Core domain contracts and evidence ownership	18
Figure-4: Durable submission and replay interaction sequence	21
Figure-5: Run lifecycle and state-transition boundaries	22
Figure-6: Local deployment processes, storage and external services	24
Figure-7: Implemented desktop prompt composer; controlled API fixture	29
Figure-8: Advanced context and retry controls in the implemented composer; fixture	30
Figure-9: Warning outcome and highlighted source in the implemented code view; fixture	31
Figure-10: Implemented evidence overview with attempt-aware inspection; fixture	32
Figure-11: Implemented metric cards with unavailable entropy; fixture	33
Figure-12: Implemented judge-evidence view showing provider failure; fixture	34
Figure-13: Implemented repair comparison and retained attempts; fixture	35
Figure-14: Implemented clarification interaction for an ambiguous request; fixture	36
Figure-15: Implemented rejected result with preserved artifact; fixture	36
Figure-16: Implemented mobile-width composer at 360 pixels; fixture	37

1. Project Title

DeHalu: Agentic Hallucination Detection and Mitigation in Local CodeLLM

1.1 Abstract

DeHalu is a software engineering project at the intersection of Software Engineering for Artificial Intelligence (SE4AI) and Artificial Intelligence for Software Engineering (AI4SE), focused on hallucination detection and mitigation in local code-generating large language models (CodeLLMs). Local models offer accessible programming assistance, but plausible output may contain invented dependencies, invalid API calls, unresolved symbols or unsupported behavioral claims. The project addresses the need to make these assertions checkable and to expose uncertainty before generated code is reused.

The methodology combines prompt normalization and clarification, local code generation, claim extraction, execution-free parsing and static analysis, package/API and symbol validation, quantitative evidence summaries, independent semantic judging, claim-level Chain-of-Verification, and a deterministic policy with bounded repair. A FastAPI backend and separate worker persist candidates and evidence in PostgreSQL, while a Next.js workspace presents progress, decisions and attempt comparisons. The implementation provides parsing adapters for eight languages, preserves partial evidence and durable history, and rechecks repaired candidates. The recorded validation comprises 64 backend, 23 frontend and eight browser tests. These controlled results demonstrate implementation behavior; they do not establish benchmark detection accuracy, live-model repair effectiveness or complete program correctness. Generated programs are not executed by the verification pipeline.

1.2 Project Overview

This overview connects the project rationale and objectives with its implemented scope, evolution, deliverables and evidence basis. Sections 1.2.1–1.2.4 explain those four aspects so that the later design and testing chapters can be read against explicit goals and boundaries.

Code-generation hallucinations require attention to source structure, external libraries and requirements, rather than fluency alone. The taxonomy survey describes code-specific categories and mitigation approaches [1], while the systematic review distinguishes hallucinations from ordinary functional errors and discusses evaluation challenges [2]. DeHalu uses these studies as domain context; the repository and recorded tests establish its delivered behavior.

1.2.1 Rationale and Objectives

A generated program can be syntactically believable, yet depend on an invented package, call a nonexistent API, reference a missing symbol, or satisfy a different requirement from the one requested. Explanatory text can reinforce these defects by presenting assumptions as facts. A developer reviewing only the final code must reconstruct those assumptions and check them independently, often without knowing which part of the response is uncertain.

Local CodeLLMs are attractive for student projects because generation can run on a user's machine and the application does not depend on a subscription to a commercial generator. Local deployment, however, does not remove hallucination risk. Model size, wording, and confidence are not substitutes for evidence about the generated artifact.

The central engineering problem is therefore to connect individual claims in generated code to explicit checks and to prevent missing evidence from becoming a passing verdict. Syntax parsing alone cannot establish API existence, package metadata alone cannot establish API compatibility, and semantic judging alone cannot override a deterministic contradiction.

Static-analysis research finds that source checks can detect some library hallucinations while leaving important limits on coverage [3]. This motivates explicit package/API evidence and uncertainty reporting rather than a claim of complete correctness. ClarifyGPT also demonstrates the relevance of targeted requirements clarification [4]; DeHalu implements its own intake questions rather than reproducing that framework.

The project objectives are to provide a usable prompt-to-evidence workflow; expose concrete hallucination-related checks; preserve uncertainty; support bounded repair; and retain an auditable history of the work performed. Table 1 connects these objectives to the implemented mechanisms and the evidence used in this report.

Objective	Implemented mechanism	Assessment evidence
O-1: Accept a natural-language programming task with optional context	Prompt composer, validated RunCreate contract, language inference and clarification	Frontend submission tests; inference and clarification tests
O-2: Detect structural and reference-related defects without executing generated programs	Tree-sitter adapters, local Semgrep rules, lexical symbol analysis, package/API catalogs	Eight-language adapter cases and deterministic verification tests
O-3: Make uncertain and contradictory evidence visible	Claim statuses, analyzer coverage, CoVe evidence links and policy explanation	Evidence-preservation tests and interface screenshots
O-4: Mitigate blocking defects through bounded repair	Evidence-grounded repair prompt, retry budget, repeated-code stop and dependency constraints	Database-backed repair and retry tests

Objective	Implemented mechanism	Assessment evidence
O-5: Preserve work independently of the browser connection	Durable job queue, worker leases, event journal and replay	Queue, cancellation, lease and history-restoration tests
O-6: Present inspectable results to the user	Code/Evidence views, selected attempts, diff, source navigation and JSON export	Component tests and responsive browser scenarios

The tests establish software behavior for controlled inputs and fixtures. They do not establish a benchmark accuracy rate for hallucination detection across arbitrary model-generated programs.

1.2.2 Scope and Evolution

The current scope follows the execution-free SRS and the implemented backend. It includes prompt intake; local Ollama generation; source and explanation claim extraction; parsing; SAST; package, symbol and API checks; quantitative evidence summaries; a three-provider judge pool; consensus; Chain-of-Verification; deterministic policy; repair; durable history; and an operational frontend.

Claim-level verification is informed by the Chain-of-Verification method [5], and independent semantic review follows the broader LLM-as-a-Judge paradigm for software artifacts [6]. These are adapted into the implemented evidence pipeline; their published results are not measurements of DeHalu.

The original proposal also discussed sandbox execution, metamorphic checks, and CrewAI orchestration. These ideas provide context for the project's evolution, but they are not described here as delivered capabilities. Generated programs are not dynamically executed as part of the committed verification pipeline, and the inspected dependency lists do not include CrewAI.

CodeHalu provides an execution-based approach to code hallucination analysis [7], while MetaQA investigates metamorphic relations and prompt mutations for hallucination detection [8]. They explain alternatives considered in the earlier proposal. Neither dynamic artifact execution nor a metamorphic-testing stage is delivered by DeHalu's final source-only pipeline.

The proposal also considered structured reasoning. Chain-of-Thought prompting introduces intermediate reasoning steps [9], but later research shows that such prompting can obscure signals used by hallucination detectors [10]. DeHalu therefore treats generated explanations and confidence as claims or optional signals, not independent proof of correctness.

Supported parsing adapters exist for Python, JavaScript, TypeScript, Java, Go, Rust, C and C++. Coverage is not identical across these languages.

Full vulnerability auditing, model training or fine-tuning, repository-wide retrieval augmentation, authenticated multi-user isolation, production hosting, and proof of program correctness remain outside this report's implemented scope. The system may issue a warning even when no blocking defect is found, because missing judges, partial coverage, unresolved claims, or repetitive output remain material limitations.

1.2.3 Deliverables and Report Organization

The delivered system consists of a FastAPI backend, an independent job worker, execution-free verification modules, PostgreSQL persistence with Alembic migrations, and a Next.js frontend. The backend tests include unit and database-backed integration coverage. The frontend includes DOM/component tests and Playwright scenarios for workflow and responsive behavior. Design artifacts and academic documents are stored under SRS and Papers.

The seven numbered chapters move from the project title, abstract and overview to Requirements Analysis, Component Level Design, Interface Design, Testing, User Manual and Conclusion. They connect the project goals to implementation, operation and recorded evidence. An unnumbered References section identifies the research papers and websites cited in the report.

Configured source repository: <https://github.com/FarhanTausif/SPL-3>.

1.2.4 Evidence Basis and Terminology

The main internal sources are SRS/DeHalu_SRS.md, SRS/High-Level-Design.md, PROJECT_RUNBOOK.md, backend/src/dehalu, frontend/app, frontend/components, frontend/lib, and the test directories. The earlier proposal is used to explain scope evolution, not to assert features absent from the current implementation. The supplied final-report cover-page PDF provides identity and submission details.

Background website material includes a general guide to hallucination detection [11] and an overview of LLM monitoring [12]. Datadog's account of structured LLM-as-a-Judge workflows provides an additional practitioner perspective [13]. These resources supply context; they do not establish DeHalu's test outcomes or transfer results from retrieval-augmented generation to code verification.

Term	Meaning in this report
Claim	A checkable assertion extracted from source, metadata or explanation, with identity and evidence status
Finding	A located analyzer observation with severity, rule identity and provenance

Term	Meaning in this report
Unsupported	An assertion contradicted by sufficiently direct available evidence
Uncertain	An assertion for which available context does not establish support or contradiction
Coverage	Whether a configured check was available, partial, unavailable or failed
Attempt	One generated or repaired artifact together with its verification evidence
Accept	The current policy's strongest result under the configured checks, not a proof of all behavior
Warn	No blocking contradiction remains, but coverage, claim, quality or judge limitations remain
Repair	Blocking evidence remains and another permitted attempt is available
Reject	Blocking evidence remains and further repair is not permitted or effective

2. Requirements Analysis

This chapter records the system's functional and non-functional requirements. Section 2.1 groups the functional obligations by pipeline stage, and Section 2.2 describes quality constraints and their acceptance evidence. Implementation notes distinguish the intended requirements from the practical coverage demonstrated in the repository.

2.1 Functional Requirements

The following requirements preserve the stable identifiers in the project SRS. They describe intended obligations, while the implementation notes after each group record the practical scope found in the repository. A requirement statement should not be read as a claim that all its possible interpretations have been experimentally validated.

2.1.1 Prompt Intake and Inference

Prompt intake turns a natural-language request into explicit task context. These requirements cover submission, inference, uncertainty and clarification before generation begins.

- FR-1.1:** Users shall submit a natural-language programming prompt.
- FR-1.2:** The system shall infer the likely target language from the prompt when the user does not state it.
- FR-1.3:** The system shall infer framework, runtime, and constraint assumptions when they are clearly implied.
- FR-1.4:** The system shall mark uncertain inferred assumptions as evidence rather than treating them as confirmed facts.
- FR-1.5:** The system shall reject or request clarification for prompts that are too ambiguous for meaningful verification.

2.1.2 Code Generation

Generation must produce an identifiable candidate and retain its provenance. The following obligations cover local model invocation, artifact storage, optional uncertainty metadata and separation of source from explanation.

- FR-2.1:** The system shall send the normalized prompt to a local CodeLLM.
- FR-2.2:** The system shall store the generated code, model identifier, prompt metadata, and generation timestamp for each run.
- FR-2.3:** The system shall capture token log-probability and predictive entropy signals when the selected model provider exposes them.
- FR-2.4:** The system shall separate generated code from explanatory text when possible.

2.1.3 Claim Extraction

Claim extraction identifies checkable assertions in source and explanation. These requirements establish claim types and stable identities before any verdict is assigned.

- FR-3.1:** The system shall extract imports, packages, APIs, symbols, functions, classes, and runtime assumptions from generated code.
- FR-3.2:** The system shall extract behavioral claims from generated explanations when they are present.
- FR-3.3:** The system shall assign each claim a stable claim identifier.

FR-3.4: The system shall classify each claim by type, including dependency, API, symbol, behavior, safety, or assumption.

2.1.4 Execution-Free Static Analysis

Execution-free analysis examines source structure and known static patterns. The requirements below specify parsing, reference checks and located findings without running generated programs.

FR-4.1: The system shall parse generated code without executing it.

FR-4.2: The system shall detect syntax and AST structure errors.

FR-4.3: The system shall detect undefined symbols where language adapters support symbol analysis.

FR-4.4: The system shall detect unsupported imports and suspicious dependency references.

FR-4.5: The system shall apply static security and quality rules to identify unsafe generated constructs.

FR-4.6: The system shall report static-analysis findings with severity, location, rule identifier, and explanation where available.

2.1.5 Tool-Backed Package and API Validation

Package and API validation checks external references against available metadata. These requirements give tool-backed evidence priority while preserving uncertainty when coverage is incomplete.

FR-5.1: The system shall validate referenced packages against configured package or symbol indexes when available.

FR-5.2: The system shall validate referenced APIs against language metadata when available.

FR-5.3: The system shall mark unverifiable package or API claims as uncertain instead of silently accepting them.

FR-5.4: The system shall use tool-backed evidence as a primary signal for dependency and API hallucination decisions.

2.1.6 Quantitative Metric Engine

The metric engine summarizes claim status, repetition and available uncertainty signals. The requirements below define reporting and comparison goals; implementation notes identify optional measurements that remain unavailable.

FR-6.1: The system shall compute Micro Hallucination Number for invalid or unsupported micro-level code claims.

FR-6.2: The system shall compute Macro Hallucination Rate for the proportion of hallucinated claims in an output.

FR-6.3: The system shall compute TR-S or an equivalent structural repetition score for repeated generated structures.

FR-6.4: The system shall correlate static invalid-reference evidence with entropy spikes when available.

FR-6.5: The system shall use metrics as evidence signals for severity ranking, policy decisions, and before/after repair comparison.

FR-6.6: The system shall expose metrics as part of the final verification report.

2.1.7 LLM-as-a-Judge Pool Verification

Independent semantic judges review aspects that deterministic source checks alone cannot establish. These requirements cover structured rubrics, consensus and the precedence of static contradictions.

FR-7.1: The system shall evaluate generated code using a pool of structured LLM-as-a-Judge evaluators.

FR-7.2: The judge pool shall include rubric coverage for requirement alignment, functional logic, code quality, safety, unsupported assumptions, dependency plausibility, and API validity.

FR-7.3: Each judge shall produce structured scores and findings instead of only free-form text.

FR-7.4: The system shall aggregate judge outputs into a consensus verdict with average score and agreement level.

FR-7.5: Judge findings shall not override deterministic static-analysis failures.

2.1.8 Chain-of-Verification

Chain-of-Verification turns extracted claims into targeted questions and evidence-linked verdicts. These requirements specify supported, unsupported and uncertain outcomes and their policy consequences.

FR-8.1: The system shall convert extracted claims into targeted verification questions.

FR-8.2: The system shall answer verification questions using static-analysis findings and tool-backed evidence.

FR-8.3: The system shall classify each checked claim as supported, unsupported, or uncertain.

FR-8.4: The system shall include unsupported and uncertain claims in the policy decision.

2.1.9 Policy Decision and Mitigation

The policy gateway combines available evidence into bounded decisions and repair. These requirements preserve blocking failures, require re-verification and limit additional candidates.

FR-9.1: The system shall combine static findings, metric scores, judge consensus, and CoVe results into a deterministic policy decision.

FR-9.2: The system shall support at least four policy outcomes: accept, warn, repair, and reject.

FR-9.3: The system shall trigger repair when blocking hallucination evidence is detected and repair is allowed.

FR-9.4: The repair agent shall receive the failed code and evidence-backed failure context.

FR-9.5: The repaired code shall pass through the verification pipeline again before being returned.

FR-9.6: The system shall limit repair attempts to prevent unbounded loops.

2.1.10 Evidence Reporting and Monitoring

Evidence reporting makes decisions and their provenance inspectable during and after a run. The following requirements cover candidate display, stage visibility, live progress and persistent history.

FR-10.1: The system shall show the final generated or repaired code to the user.
FR-10.2: The system shall show the policy decision and hallucination risk evidence.
FR-10.3: The system shall display stage-level results for generation, claim extraction, static analysis, metrics, judge verification, CoVe, policy, and repair.
FR-10.4: The system shall persist run history and evidence for later review.
FR-10.5: The frontend shall provide a live or near-real-time view of the verification workflow when backend events are available.

2.2 Non-Functional Requirements

Table 2 preserves the SRS's non-functional categories. Acceptance evidence is tied to available tests and code inspection. Performance thresholds not specified in the source are not replaced by invented latency targets.

ID / Category	Requirement	Assessment evidence / limit
NFR-1 / Privacy	The system shall support local CodeLLM generation so user prompts and code can remain on the user's machine when configured locally.	Local generation supported; cloud judges transmit selected material externally, so fully local privacy is conditional.
NFR-2 / Reliability	The system shall fail closed by warning or rejecting output when blocking verification stages fail or evidence is insufficient.	Policy warns for incomplete checks and rejects unresolved blockers; failure/partial evidence is retained.
NFR-3 / Explainability	The system shall provide evidence for each accept, warn, repair, or reject decision.	Claims, findings, source links, coverage and policy reasons are visible and exportable.
NFR-4 / Extensibility	The system shall allow additional analyzers, package registries, model providers, and judge rubrics to be added without rewriting the whole pipeline.	Provider/catalog/analyzer boundaries support extension; complete support for every future tool is not claimed.
NFR-5 / Compatibility	The frontend shall support modern desktop browsers and responsive layouts for common laptop screen sizes.	Browser scenarios passed at four widths in Chromium; wider cross-browser support remains to be tested.

3. Component Level Design

This chapter explains how DeHalu’s components cooperate to generate, inspect and retain code evidence. Its seven subsections cover the overall architecture, component responsibilities, domain contracts, metrics and policy rules, behavioral workflows, API/event contracts and local deployment. Together they connect the requirements to the system’s operational boundaries.

3.1 Overall System Design

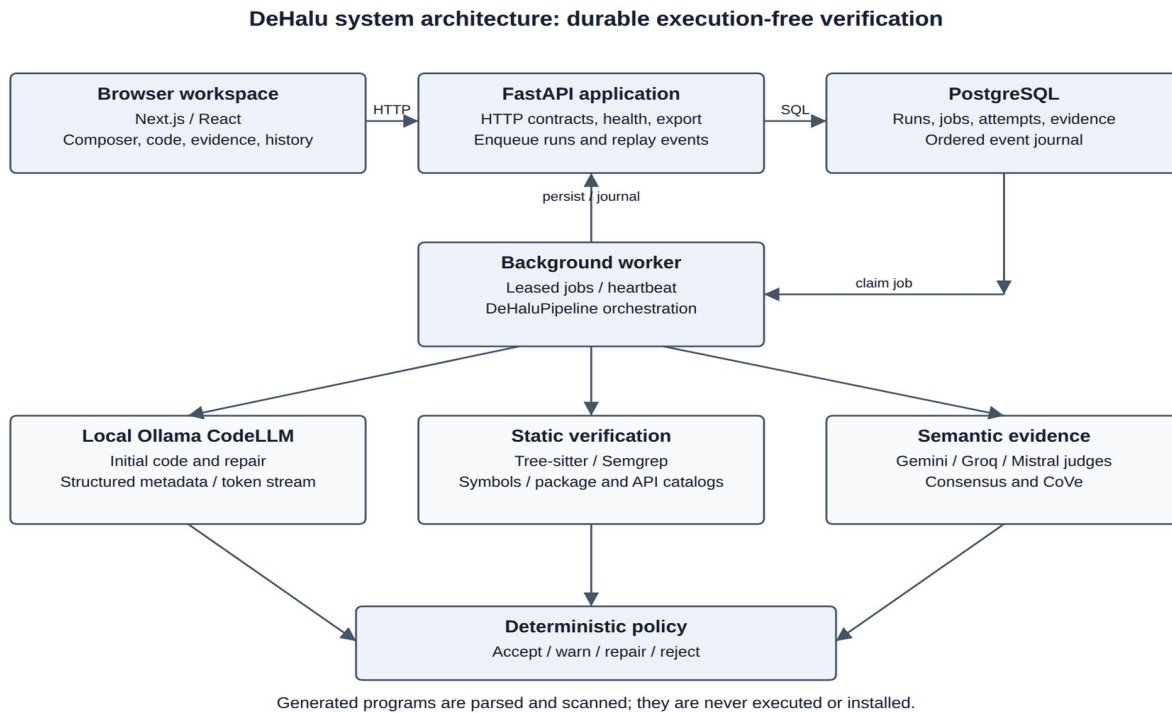


Figure-1: Overall system architecture and verification boundary

Figure 1 presents the main components and their persistence boundary. The browser communicates with the FastAPI application for submission, snapshots, evidence, health, cancellation, clarification and event replay. Submission persists a job instead of tying its execution to the lifetime of an HTTP request. A separate worker claims the job and drives the verification pipeline.

PostgreSQL serves both as the evidence store and the durable queue/event journal. The worker calls the local generator, runs source-only analyzers, requests semantic judgments, and writes checkpoints and completed attempts.

The design separates transport models from verification logic and persistence models. Pydantic contracts define the shape of requests and evidence; SQLAlchemy records define stored relationships; and analyzer/provider modules produce typed evidence consumed by the service layer.

The strongest verification boundary is execution-free. The application launches trusted analysis tools against generated source, but it does not import generated modules, install their declared packages, or run their programs.

Figure 2 shows the normal verification order and bounded feedback loop. A repair is another candidate artifact and must pass through extraction and verification again.

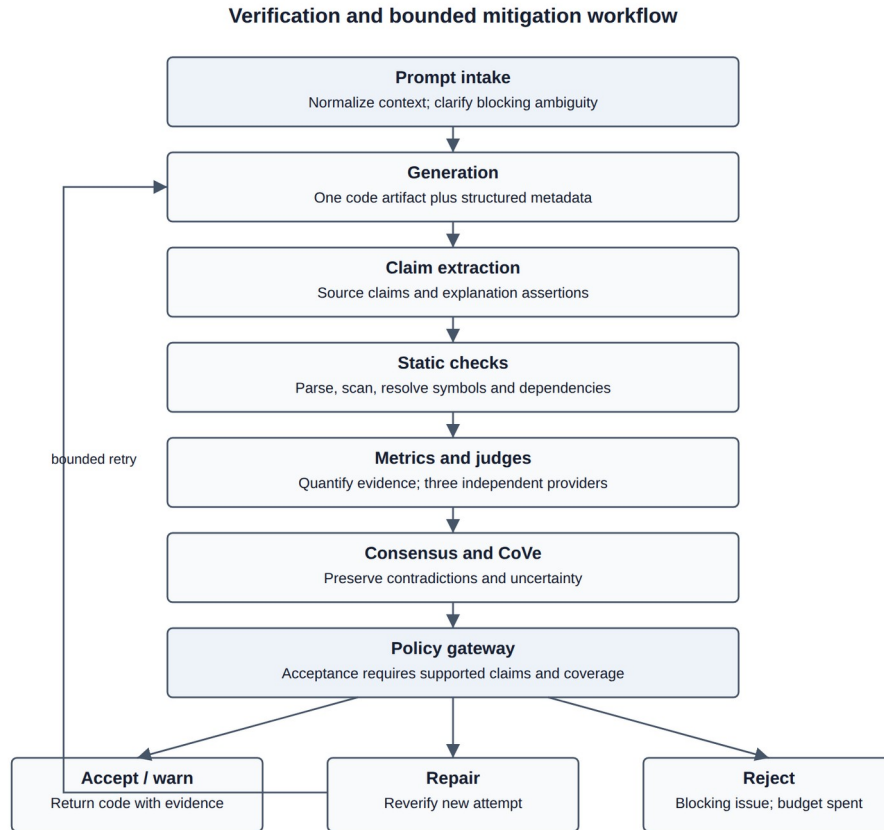


Figure-2: Verification stages and bounded repair workflow

3.2 Component-wise High-Level Design

This section describes the twelve major components from prompt composition and API intake through generation, static verification, semantic review, policy/repair and persistence. Each component subsection identifies its responsibilities, collaborators, data contracts and failure behavior so that the overall architecture can be traced to implementation modules.

3.2.1 Prompt Composer and Workspace Controller

RunComposer converts the natural-language task and advanced options into RunCreate. It owns prompt, language, framework, runtime, libraries, constraints and repair settings. The client trims input, parses comma-separated libraries and newline-separated constraints, and prevents short submissions; server validation remains authoritative.

useWorkspace owns selected run, attempts, streamed drafts, history, stages and connection status. Its watcher cancellation/request epoch prevents stale responses from replacing a newly selected run. Historical attempt selection stays stable when newer evidence arrives. New run starts a draft; cancellation is a distinct server action. Network failure changes connection feedback without inventing a failed policy. Collaborators include the API client, CodeAttempts and EvidencePanel. Source: frontend/lib/use-workspace.ts and components/RunComposer.tsx.

3.2.2 API Boundary and Domain Contracts

FastAPI routes under /api validate RunCreate and ClarificationAnswers and expose RunSummary/RunEvidence. RunCreate allows prompts of 3–30,000 characters and explicit maximum repairs of 0–5. POST /runs enqueues durable work and returns HTTP 202 instead of performing generation within the request.

The boundary serves history, snapshots, evidence, export, cancellation, clarification, health and event replay. Missing runs return 404, malformed UUIDs produce validation errors, invalid clarification state returns 409 and invalid replay IDs return 400. Shared Pydantic schemas generate frontend contracts. The prototype has no completed authenticated ownership layer; run IDs and session records do not provide access control. Source: api/routes.py and domain/models.py.

3.2.3 Durable Job Worker and Run Orchestrator

The workflow module creates jobs, claims queued work, maintains leases, cancels runs and journals ordered events. Row locking with skip_locked permits concurrent workers to claim different jobs while preventing duplicate ownership. Heartbeats extend an owner's lease; expired jobs become interrupted rather than silently completed.

DeHaluPipeline orders intake, generation, extraction, analysis, metrics, judges, consensus, CoVe, policy and repair. It delegates checks to typed modules and saves partial checkpoints/completed attempts. Guarded writes stop cancelled/interrupted work; infrastructure failures retain available evidence. The separate worker uses configurable thread-pool concurrency. This implemented Python engine supplies agentic responsibilities without a CrewAI dependency. Source: services/pipeline.py, state/workflow.py and worker.py.

3.2.4 Local Generator and Artifact Normalizer

OllamaClient streams candidates from the configured local /api/generate endpoint. The artifact validator requires one nonempty primary fenced block with compatible language and JSON metadata containing assumptions, dependencies, entry_points and limitations. Optional repair metadata identifies changes, fixed evidence and remaining uncertainty.

The local streaming generation interface follows Ollama's documented Generate API [14]. DeHalu's single-artifact validation and metadata requirements are application-level constraints rather than guarantees supplied by the model runtime.

Empty/multiple artifacts, language conflicts and incorrectly typed metadata are rejected. Missing required metadata can be normalized separately without changing source; provenance records the incomplete original. Provider retries are bounded. Missing entropy/log-probability data remains unavailable, and fake generation is explicit test provenance. Model names are deployment configuration, not experimentally established recommendations. Source: `providers/llm.py` and pipeline artifact handling.

3.2.5 Claim Extraction and Language Adapters

The adapter registry supports Python, JavaScript, TypeScript, Java, Go, Rust, C and C++ parsing/extraction. Located source assertions become Claim records with UUID, type, text, `source_range`, checking method and evidence fields. Dependency metadata can add claims, while live structured extraction adds explanation-derived behavior, runtime, assumption and safety assertions.

Extraction identifies assertions rather than proving them. An “all edge cases” explanation remains unverified until evidence supports it. Failures mark incomplete coverage, and isolated adapters preserve language-specific behavior. Tests cover representative valid/malformed samples across all eight languages; they do not establish complete semantics. Source: `verification/adapters.py` and `claims.py`.

3.2.6 Static Parser and SAST Executor

Tree-sitter reports source structure errors, while Semgrep scans a temporary file with a language-appropriate extension using local repository rules. The scanner uses explicit configuration, JSON results, disabled telemetry/version checks and a 45-second timeout. Generated programs are never imported, installed or executed.

Tree-sitter supplies source parsing [15], while Semgrep supplies configurable static rule matching [16]. Their tool capabilities explain the analyzer choices; their presence alone does not prove that all hallucinations or vulnerabilities are detected.

Findings carry rule, severity, message, provenance and source ranges. Coverage separately records available, partial, unavailable or failed checks, so an empty finding list is not evidence that scanning succeeded. Known risky patterns are limited SAST coverage, not a security audit. Warning-level shell/process/deletion operations can receive task-aware semantic review. Source: `static_analysis.py` and `rules/core.yaml`.

3.2.7 Symbols, Packages and API Catalogs

Symbol validation resolves supported lexical bindings, aliases, parameters, nested scopes and shadowing. Selected API catalogs check positional arity/keywords; a complete enough catalog can contradict an invented member. Partial catalog absence, unresolved project context and unknown submodules remain uncertain.

Optional registry requests distinguish an authoritative 404 from an outage. A successful package lookup supports existence but not API compatibility. Results are cached, with unavailable evidence cached briefly. Python has deeper supported resolution than some other adapters; no equal semantic coverage is

claimed. A suspicious package name alone does not prove nonexistence. Source: `verification/symbols.py` and `catalogs.py`.

3.2.8 Metric Engine

`compute_metrics` summarizes code, claims, findings and available entropy metadata in versioned `MetricResult`. MiHN counts unsupported claims; MaHR includes unsupported plus uncertain/not_checked claims divided by total. With no claims, MaHR is zero but uncertainty is one. These values do not establish correctness or a calibrated probability of hallucination.

TR-S measures repeated meaningful lines and nonoverlapping 12-token blocks, rather than complete AST equivalence. Severity and available entropy contribute to a weighted heuristic risk score. Metrics aid comparison, while deterministic policy blockers remain authoritative. Section 3.4 gives exact formulas. Source: `verification/metrics.py`.

3.2.9 Judge Pool and Consensus

`JudgePool` requests independent structured review from Gemini, Groq and Mistral for requirement alignment, functional logic, quality/safety, dependency/API plausibility, assumptions and risk. Results retain model, status, optional verdict/score, rubric, explanation, blocking issues and evidence references. Missing credentials or failed calls do not fabricate support; simulation is not a valid judge response.

Consensus uses only ok results with verdict and score. Two failing judges yield fail; all three valid judges must pass for pass; otherwise consensus warns. The average uses valid scores and is null when none exist. Configured credentials are not proof of access/quota. Reviews remain fallible, execution-free semantic signals. Source: `providers/llm.py` and `verification/policy.py`.

Research on software-engineering judges identifies the need for reliable, multi-faceted artifact evaluation and the limitations of current approaches [6]. The saved Datadog article describes rubric-based structured judging in a different, RAG-focused setting [13]; DeHalu does not claim those published results for its own judge pool.

3.2.10 Chain-of-Verification

`run_cove` asks a targeted verification question for each claim. Linked error findings preserve unsupported verdicts, while a claim without direct supporting evidence remains uncertain. Structured semantic review can inspect normalized requirements and source evidence, without executing the artifact.

`merge_semantic_checks` validates incoming evidence identifiers and prevents a semantic support assertion from overriding static contradiction. `CoVeResult` records question, claim, verdict, evidence, references and confidence; confidence cannot replace supporting evidence. This is an execution-free adaptation of verification questions, not an assumption that self-questioning establishes truth. Source: `verification/cove.py`.

The original CoVe method separates drafting from verification questions and answers [5]. DeHalu adapts that idea to stored code claims and direct tool evidence, retaining unsupported contradictions and uncertainty rather than relying on self-verification alone.

3.2.11 Policy Gateway and Repair Controller

Error findings, unsupported CoVe or majority semantic failure block acceptance. Policy repairs when allowed and otherwise rejects. Without blockers it warns for incomplete coverage, uncertain/absent CoVe, fewer than three valid judges, nonpass consensus, warning findings or TR-S at least 0.3. Accept requires these limitations to be absent.

Repair receives the failed artifact, task and evidence, then produces a newly validated and reverified candidate. The initial output is attempt one; a budget of three allows up to three additional repairs. Repeated code hashes stop ineffective cycling. Unknown fixed-evidence IDs are rejected and unverified new dependencies can become blocking findings. Source: `verification/policy.py` and `services/pipeline.py`.

3.2.12 Persistence and Evidence Repository

RunRepository persists and reconstructs run lifecycle, generated candidates and linked evidence. Relational records support ownership/foreign keys; JSONB payloads retain richer transport fields and partial checkpoints. Attempt identity is unique within a run, and CoVe links preserve stored claim identities.

Jobs/events are separate from outputs: zero-attempt runs may still contain clarification, cancellation or failure evidence. RunEvidence exposes complete attempts and available partial state for UI/export. The configured PostgreSQL volume retains data until operator maintenance; automatic retention and authenticated user isolation are not delivered features. Source: `state/repository.py` and `state/models.py`.

3.3 Domain and Design Classes

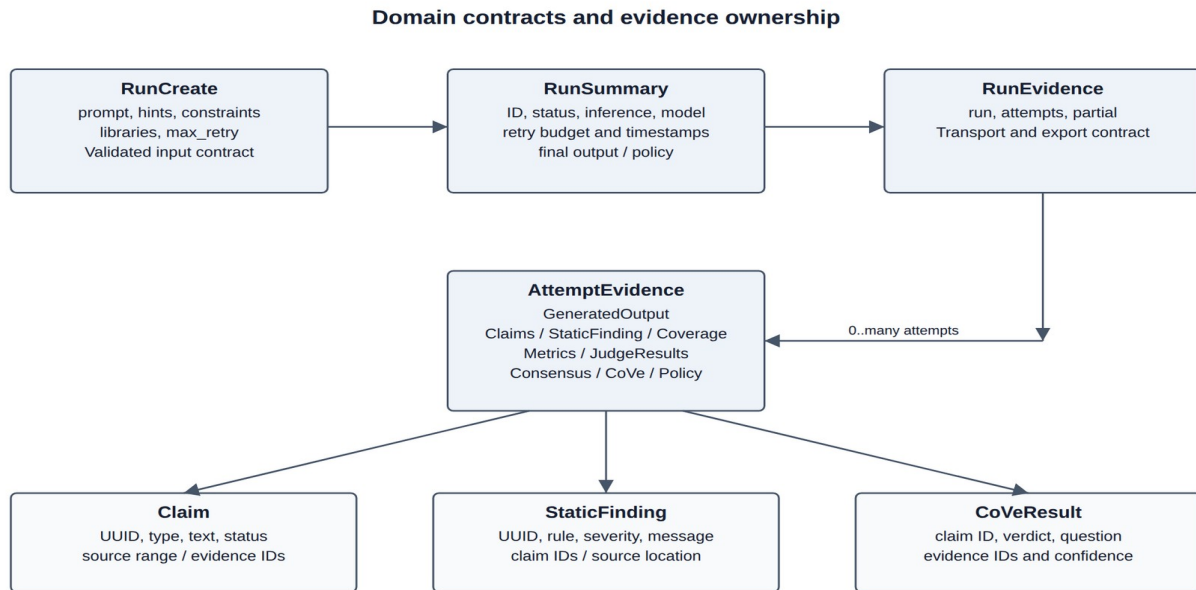


Figure-3: Core domain contracts and evidence ownership

Figure 3 shows the transport-independent contracts that connect intake, execution and reporting. RunCreate is input; RunSummary describes lifecycle; RunEvidence groups complete and partial results; and AttemptEvidence owns a candidate's verification bundle. Claims and findings remain separate because an assertion and an analyzer observation have different roles.

Design class	Responsibility	Principal collaborators
DeHaluPipeline	Stage ordering, checkpoints, artifact evaluation and repair loop	Repository, generator, judges, analyzers, policy
OllamaClient	Local streaming/structured generation and bounded provider handling	Settings, artifact contracts, pipeline
JudgePool	Structured independent provider review	Settings, JudgeResult, consensus function
LanguageAdapter	Parser, extraction and advertised coverage for a language	Adapter registry, claims, static analysis

Design class	Responsibility	Principal collaborators
RunRepository	Store/reconstruct runs, attempts and linked evidence	SQLAlchemy session and record classes
RunCreate / InferenceResult	Requested and normalized task context	API validation, inference, generator prompt
Claim / StaticFinding	Assertions, contradictions and source locations	Symbol checks, SAST, CoVe, interface
MetricResult / PolicyDecision	Quantitative summaries and rule-based outcome	Metric engine, policy gateway, report UI
AttemptEvidence / RunEvidence	Versioned candidate evidence and run-level export	Pipeline, repository, API, frontend

The table describes implemented classes and contracts, not a requirement that every conceptual component be an object-oriented class. Several deterministic steps, including metrics, policy and consensus, are plain functions.

3.4 Metrics and Decision Rules

Let N be the number of extracted claims, U the number marked unsupported, and Q the number marked uncertain or not_checked. The metric implementation recorded in this report uses the following definitions:

```

MiHN = U
MaHR = (U + Q) / N, if N > 0; otherwise 0
Uncertainty = Q / N, if N > 0; otherwise 1
Severity = min(1, (errors + 0.3 × warnings) / 5)
TR-S = max(repeated-line fraction, repeated 12-token-block fraction)
Risk = min(1, 0.4 × MaHR + 0.2 × TR-S
           + 0.3 × Severity + 0.1 × available_entropy)

```

Unavailable entropy is stored as null and contributes no measured entropy term in the current risk expression. This choice is a limitation of the heuristic, not evidence that model uncertainty was zero. The interface consequently shows “Unavailable” rather than displaying null as a measured 0.00.

For an illustrative arithmetic calculation, two unsupported claims and one uncertain claim among ten produce MiHN 2 and MaHR 0.30. This is a worked explanation of the formula, not a reported experiment. A warning outcome may still occur at a low risk score because incomplete coverage or missing judges are independent policy gates.

Evidence condition	Policy response	Rationale
Error finding, unsupported CoVe, or at least two valid failing judges	Repair if allowed; otherwise reject	A blocking contradiction or semantic failure remains
No blocker, but uncertain claims, missing/failed checks or incomplete/disagreeing judges	Warn	Available evidence is insufficient for unrestricted acceptance
No blocker, but warning finding or TR-S at least 0.3	Warn	Quality/risk limitations remain visible
Material claims supported, configured coverage available, all three valid judges pass and no other warning gate	Accept	Configured verification policy is satisfied
Ambiguous input with specific unanswered questions	Needs clarification workflow	A useful candidate cannot yet be generated from known requirements

The policy deliberately does not use a single global risk percentage as its acceptance threshold. Scores are comparative aids. They should be interpreted alongside claim types, direct contradictions, tool coverage and provider availability, especially when comparing repaired and initial outputs.

3.5 Behavioral Design

Behavioral design explains how the components interact over a run's lifetime. Section 3.5.1 follows submission, queue processing and event replay; Section 3.5.2 describes lifecycle transitions and failure preservation; and Section 3.5.3 explains repair and evidence provenance. These interactions retain a consistent relationship between a run, its candidates and the checks performed.

3.5.1 Submission, Queue and Replay

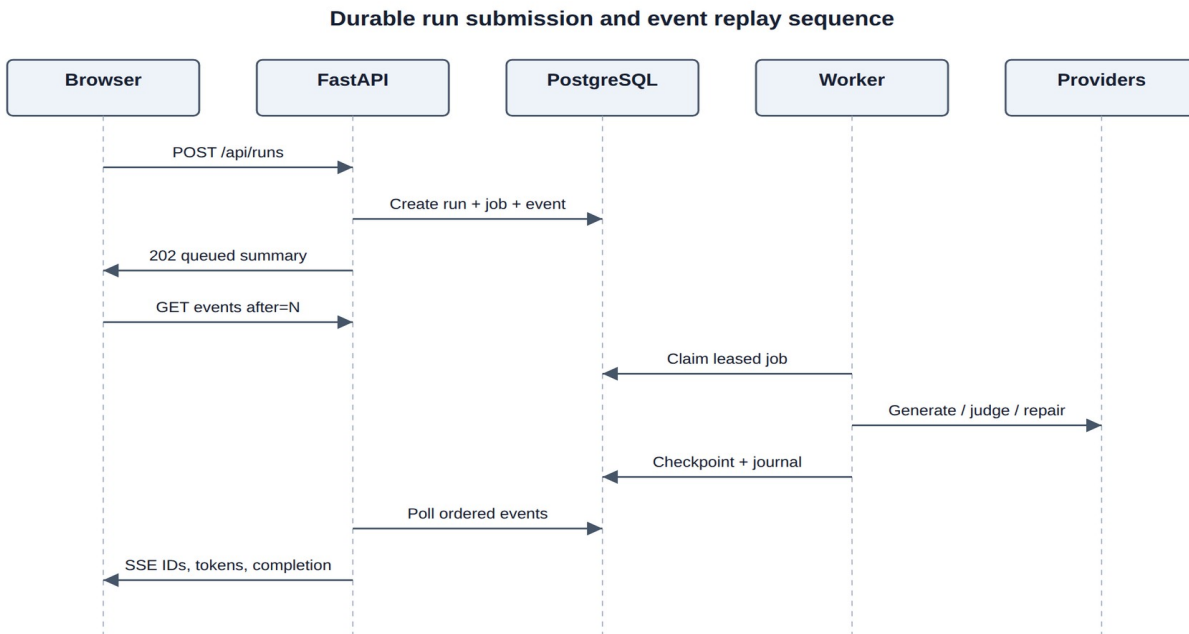


Figure-4: Durable submission and replay interaction sequence

Figure 4 shows the principal message sequence. `POST /runs` returns once the request has been persisted and queued. The worker acquires an owned job separately; it does not depend on the browser retaining the `POST` request. Generation and verification checkpoint into PostgreSQL, while the API event endpoint reads ordered journal entries.

The event endpoint accepts an after cursor and Last-Event-ID header, using the greater valid nonnegative sequence. It streams events with SSE identifiers and periodically emits heartbeats. Terminal runs drain available events before closing. The former `POST /runs/stream` endpoint remains a compatibility wrapper around the same queue rather than a separate execution engine.

3.5.2 Run Lifecycle and Failure Preservation

Figure 5 separates queued/running states, waiting clarification, and terminal outcomes. Successful workflow completion can mean accept or warn; “completed” is not synonymous with “accepted.” Rejected work retains its candidates and evidence. Failed and interrupted work retains partial checkpoints where available.

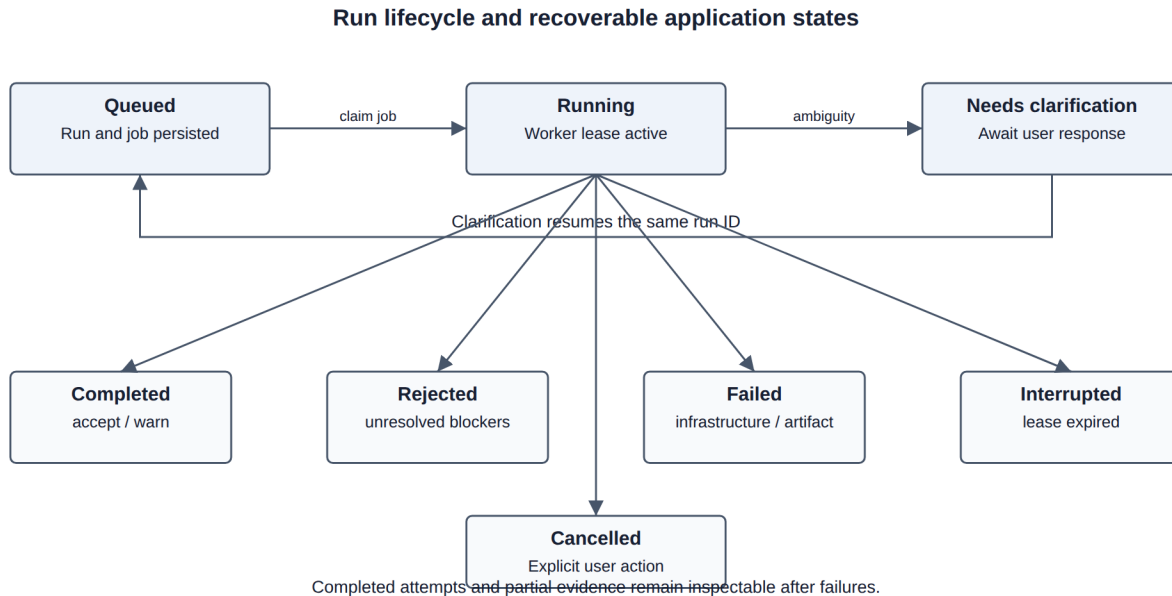


Figure-5: Run lifecycle and state-transition boundaries

Cancellation and lease expiration protect against stale writes by making the orchestrator stop when it reaches a checkpoint or event boundary. Clarification resumes the existing job and request in a controlled state. Expired work is marked interrupted and is not automatically retried with invented semantics. These choices favor an inspectable failure over hidden repetition.

3.5.3 Repair and Evidence Provenance

A repair request is constructed from the latest failed candidate, not merely from the original prompt. The workflow includes source findings, claim verdicts, provider feedback and specific CoVe contradictions. A new artifact must respect the normalized context and dependency constraints. Its metadata cannot claim to fix evidence identifiers that do not belong to the prior candidate.

After repair, extraction and verification repeat. The earlier attempt remains stored; the frontend can inspect its code, evidence and result. A repeated SHA-256 code digest disables another ineffective repair, and the policy still rejects if blocking evidence remains. This prevents an unchanged artifact from gaining acceptance merely because time or retries have elapsed.

3.6 API and Event Contracts

This section summarizes the HTTP and event contracts shared by the backend and frontend. The endpoint table identifies each operation's request/response and purpose, followed by an explanation of event identity and health reporting. These contracts separate transport status from verification conclusions.

Operation	Contract / response	Purpose
GET /api/health	HealthResponse	Database, generator, configured judge and analyzer capability visibility
POST /api/runs	RunCreate → RunSummary; HTTP 202	Persist and enqueue work
GET /api/runs	List of RunSummary; bounded limit/offset	Recent history
GET /api/runs/{id}	RunSummary	Current or terminal lifecycle snapshot
GET /api/runs/{id}/evidence	RunEvidence	Complete attempts and partial stage data
GET /api/runs/{id}/export	JSON attachment containing RunEvidence	Download audit evidence
POST /api/runs/{id}/cancel	RunSummary	Stop active or waiting work
POST /api/runs/{id}/clarification	ClarificationAnswers → RunSummary; HTTP 202	Resume waiting clarification
GET /api/runs/{id}/events	text/event-stream; after / Last-Event-ID	Durable event replay and follow
POST /api/runs/stream	RunCreate → text/event-stream	Compatibility submission/stream wrapper

Event payloads carry run_id, sequence and timestamp. Typed events include run_created, stage, token, context, clarification, attempt_completed, run_completed, run_cancelled, run_resumed, run_interrupted and error.

The health endpoint distinguishes reachability/configuration from actual provider readiness. Configured judge credentials are labeled `configured_unchecked` until an observed attempt supplies a result. A degraded health status should therefore be used to diagnose missing capability, not interpreted as evidence that every operation will fail.

3.7 Deployment Design

Figure 6 shows the supported local deployment. The browser loads Next.js on port 3000. FastAPI runs on port 8000; the separate Python worker consumes queued jobs; Docker exposes PostgreSQL port 5433 on the host; and Ollama normally listens on 11434. External semantic judges are reached through HTTPS when configured.

The API and worker must share the same database and model/provider configuration. Alembic creates/upgrades the schema before normal operation.

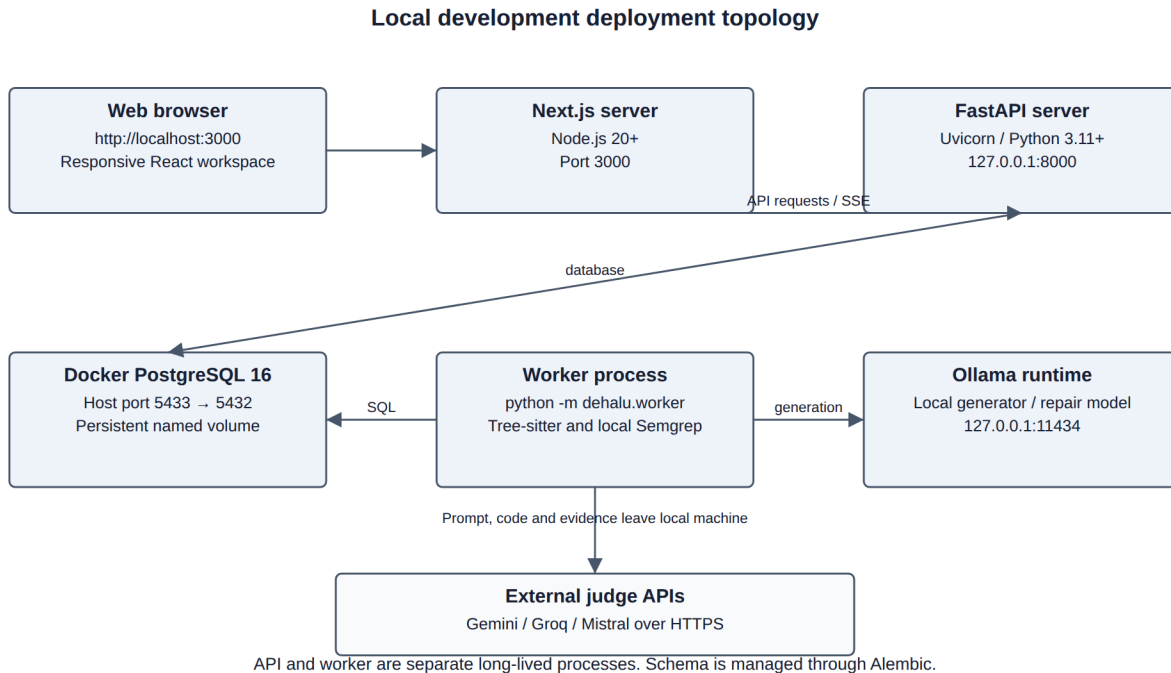


Figure-6: Local deployment processes, storage and external services

The frontend's `NEXT_PUBLIC_API_BASE_URL` must match the reachable API, and CORS settings must permit the chosen browser origin. Configuration files remain local and secrets are not committed.

4. Interface Design

This chapter describes DeHalu's implemented user interface and the feedback it gives during verification. Its five subsections cover navigation, interface objects and actions, state-changing events, actual screen

examples, and responsive/accessibility considerations. The screenshots retain their controlled-fixture provenance.

4.1 Users, Tasks and Navigation

The interface is organized as a workspace rather than a sequence of disconnected forms. A desktop history sidebar supports run discovery and new drafts. The main area begins with a prompt composer and becomes an inspectable run view after submission. Header controls show status, provider coverage, cancellation where applicable and export. A compact workflow strip exposes grouped verification stages.

The primary views are Code and Evidence. Both share an attempt selection so that a claim or policy result does not accidentally refer to a different artifact than the one being inspected.

The user flow is: compose a task; supply optional context; submit; resolve clarification if requested; follow generation and checks; read the policy reason; inspect relevant evidence; compare repair attempts if present; and copy or export only after considering limitations. Starting a new draft and cancelling an existing run are separate actions with different consequences.

4.2 Interface Objects and Actions

Interface objects connect user actions to observable system feedback. The following table maps the composer, inspection controls, evidence navigation and run-management actions to their results; the accompanying discussion explains status communication and failure feedback.

Object or control	User action	Result and feedback
Prompt composer	Enter a task or choose an example	Editable natural-language request; minimum-length submission check
Advanced options	Set language, repairs, framework, runtime, libraries and constraints	Context is included in the request without replacing the prompt
Generate	Click or use Ctrl/Cmd+Enter	Pending submission feedback followed by queued/run workspace
Workflow stage group	Expand stage details	Status/progress and candidate context are inspectable

Object or control	User action	Result and feedback
Inspect selector	Choose initial, historical or latest attempt	Code, policy and evidence target the same candidate
Code / Changes tabs	Read source or compare to the initial artifact	Syntax-highlighted code and line-based differences
Evidence tabs and filters	Select category, search and status	Located claims/findings and structured review data
Evidence/source links	Open linked claim/finding or code line	Focused evidence or highlighted source location
Provider status	Open the status popover	Configured versus observed availability and failure information
History sidebar	Search/open/refresh or start New run	Saved work selection or a fresh composer
Cancel	Stop active/waiting work	Explicit cancellation while retained evidence remains available
Export	Download the current run's evidence	JSON audit artifact from the backend

The design uses text labels and status badges in addition to color. A warning banner explains why the outcome is limited. The copy action handles a clipboard failure visibly; it does not imply the program was executed or validated dynamically.

4.3 Events and State Transitions

This section relates user and backend events to visible interface transitions. The state table covers submission, clarification, generation, completion, reconnection, cancellation and history restoration, making the distinction between connection status, job lifecycle and policy outcome explicit.

Current interface state	Event	Next visible state / behavior
Empty composer	Valid submit	Pending submission, then queued workspace
Queued workspace	Worker stage event	Running stage progress and candidate context
Running generation	Token event	Matching attempt's source grows in the code view
Intake	Clarification required	Questions and answer control; same run retained
Waiting clarification	Valid answer accepted	Requeued run, then updated intake/generation
Verifying	Attempt completed	Candidate code/evidence saved; repair may follow
Active run	Run completed	Final accept/warn/reject policy and retained attempts
Any open run	Select historical attempt	Candidate selection remains stable as new events arrive
Active run	Browser connection lost	Reconnecting/offline connection message; server job continues

Current interface state	Event	Next visible state / behavior
Running job	Explicit Cancel	Cancelled state; completed/partial evidence preserved
Running job	Lease expires	Interrupted status with recovery explanation
Any workspace	New run	Fresh composer; background work is not cancelled
Saved run	Reload/open from history	Snapshot restored; event following resumes if needed

4.4 Interface Screens and Evidence Provenance

The following images show the implemented Next.js application rendered in Chromium. They were captured on 8 October 2026 using controlled API/SSE fixtures so the displayed states are reproducible without paid provider calls. Fixture verdicts, dates, example code and provider messages illustrate the UI; they are not observations of a live generator or judge. The software validation results are reported separately in Section 5.

4.4.1 Initial Prompt Workspace

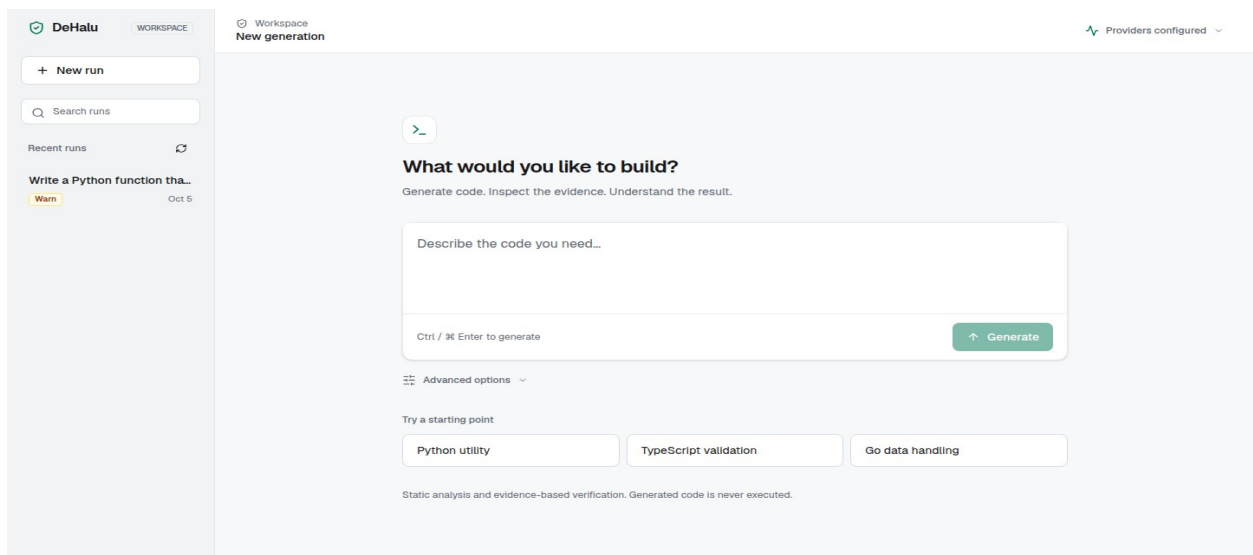


Figure-7: Implemented desktop prompt composer; controlled API fixture

Figure 7 shows the desktop composer. The screen gives a single central prompt area, visible Generate action, optional examples and a persistent history area. It avoids requiring model-provider choices for every request. The provider-status control can indicate limited coverage before the user begins.

4.4.2 Advanced Context

Figure 8 shows the advanced form. Language selection can override automatic inference. Maximum repairs controls additional candidates, not the total including initial generation. Framework/runtime fields and explicit constraints reduce avoidable ambiguity. Leaving them blank does not authorize the model to invent context without recording assumptions.

Figure-8: Advanced context and retry controls in the implemented composer; fixture

4.4.3 Code and Warning Outcome

Figure-9: Warning outcome and highlighted source in the implemented code view; fixture

Figure 9 shows a completed fixture run with a warning. Its arithmetic source deliberately contains a questionable operation for evidence-navigation testing. Selecting a linked source location highlights the relevant line. The banner preserves the policy limitation while Code and Evidence remain available.

4.4.4 Evidence Overview

Figure 10 shows the Evidence workspace. Status, coverage and counts are grouped separately from the generated code. Tabs organize detailed assertions, analyzer observations, metrics, judges and verification questions.

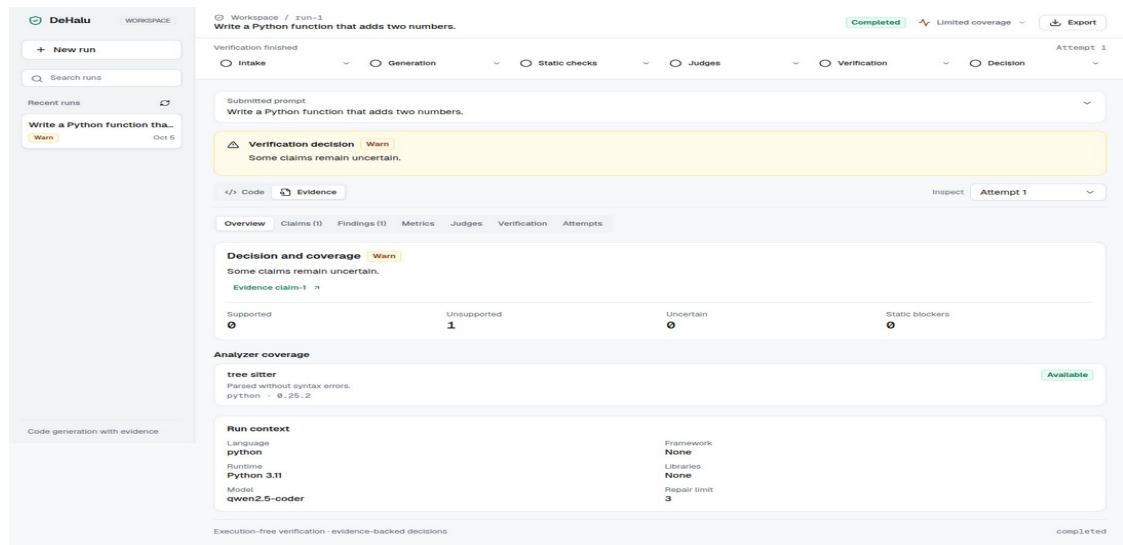


Figure-10: Implemented evidence overview with attempt-aware inspection; fixture

4.4.5 Metrics and Missing Measurements

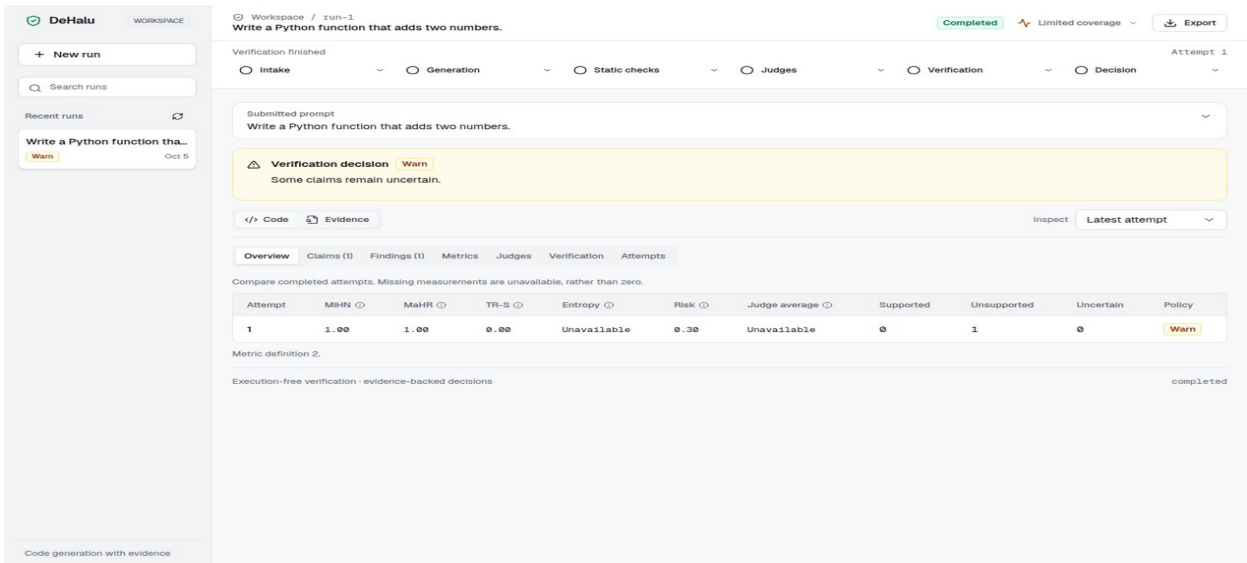


Figure-11: Implemented metric cards with unavailable entropy; fixture

Figure 11 shows numeric evidence alongside an unavailable entropy measurement. The UI formats missing values as unavailable, and metric help describes their interpretation. MiHN is a count, MaHR includes unresolved claims, and risk is a heuristic comparison score rather than a probability of incorrect code.

4.4.6 Provider Failure and Judge Coverage

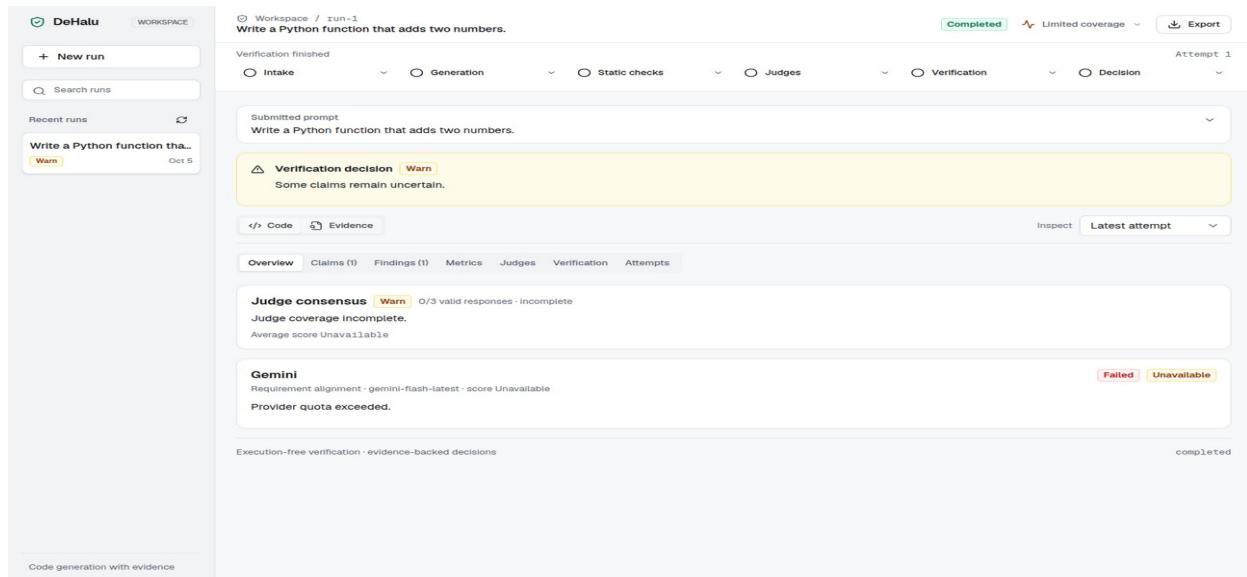


Figure-12: Implemented judge-evidence view showing provider failure; fixture

4.4.7 Repair Comparison

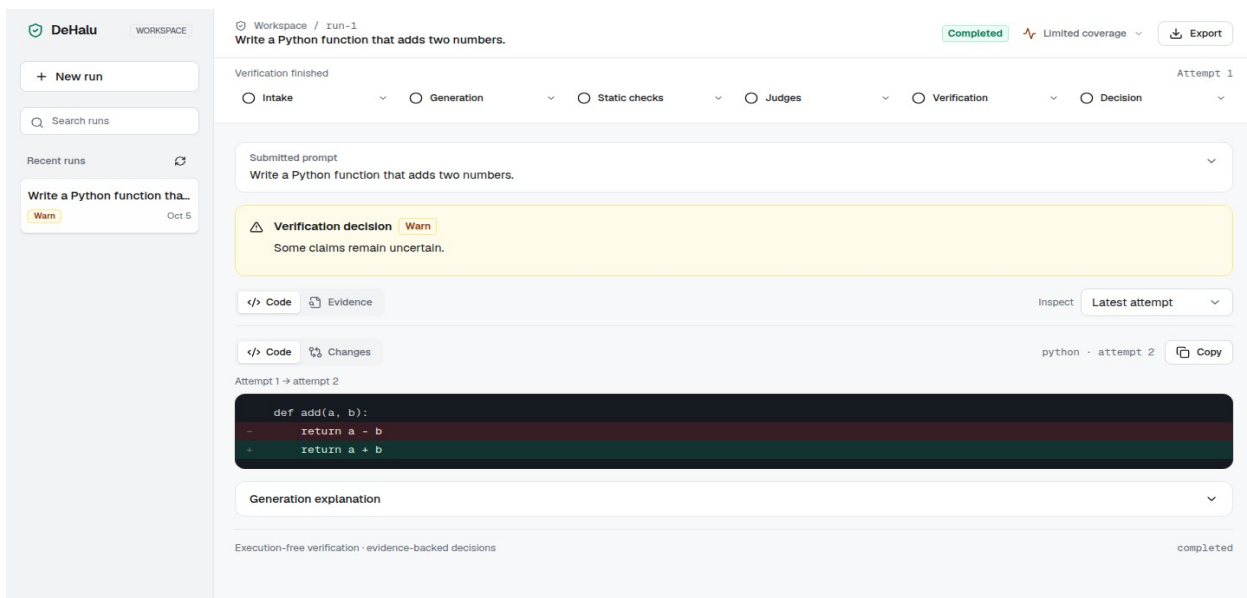


Figure-13: Implemented repair comparison and retained attempts; fixture

4.4.8 Clarification and Rejection

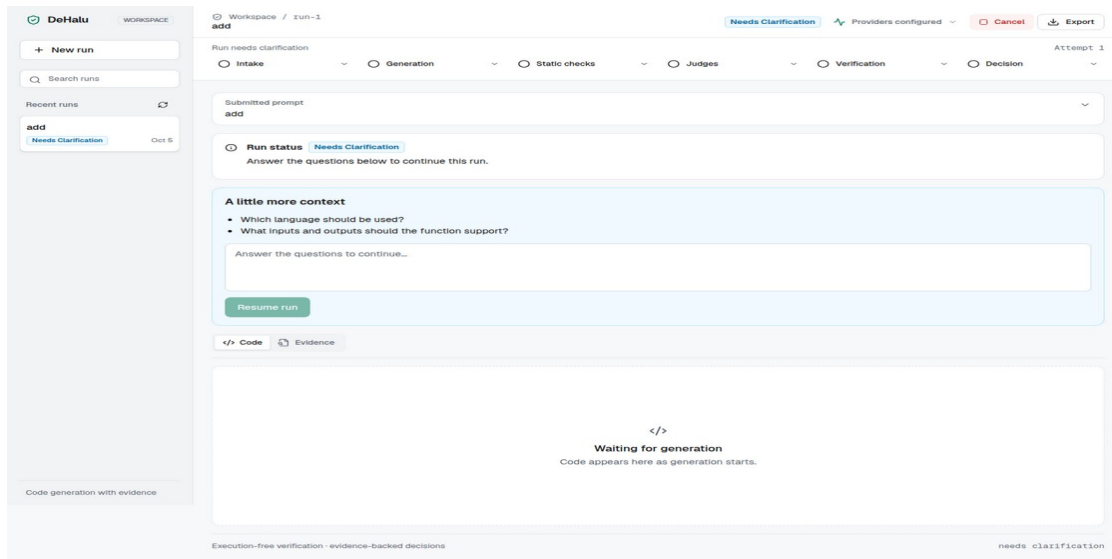


Figure-14: Implemented clarification interaction for an ambiguous request; fixture

Figure 14 shows a waiting clarification state. The questions correspond to missing task context and permit the user to continue the existing run. Figure 15 shows a rejected outcome whose artifact remains inspectable, allowing the user to understand the unresolved issue rather than receiving only a failure notification.

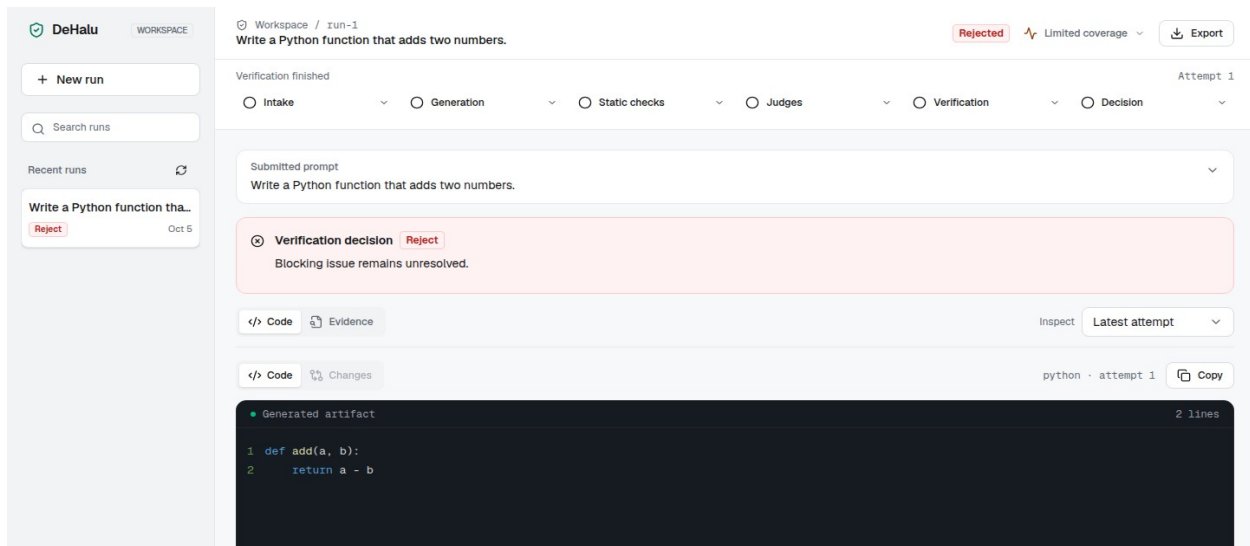


Figure-15: Implemented rejected result with preserved artifact; fixture

4.5 Responsive and Accessibility Considerations

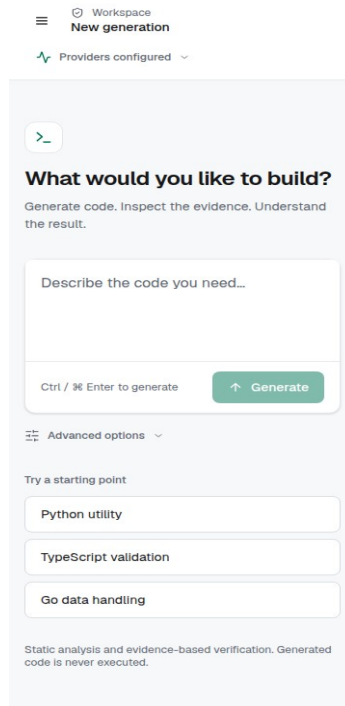


Figure-16: Implemented mobile-width composer at 360 pixels; fixture

The desktop layout reserves a sidebar for history, while narrower screens use a history drawer. Controls wrap and evidence tabs can scroll horizontally within their own container. The browser checks, verify that the document does not overflow horizontally at widths of 360, 768, 1280 and 1440 pixels. Figure 16 illustrates the narrow-screen composer.

The interface includes labeled controls, tab/combobox semantics, status/alert roles, bundled Geist fonts and a keyboard submission shortcut. Source links provide visible highlighting, and reduced-motion preferences affect navigation animation. The tests confirm selected interaction and font-loading behaviors; they do not constitute an exhaustive accessibility audit or a formal compliance certification.

Useful further assessment includes keyboard-only navigation across long evidence tables, screen-reader testing of changing stage statuses, and readability checks for long provider messages. The report treats these as refinements rather than claiming unperformed validation.

5. Testing

This chapter presents the testing strategy and recorded validation evidence. Its eight subsections define the approach and pass/fail criteria, summarize observed results, detail backend and frontend/browser cases, discuss defects and evaluation limits, identify risks, and provide reproduction guidance. The evidence concerns controlled software checks rather than a live-model effectiveness benchmark.

5.1 Testing Approach and Strategy

Testing addresses four levels: deterministic backend units, PostgreSQL-backed contracts/workflow, frontend state/rendering units, and browser interactions. The strategy prioritizes observable behavior: a contradiction cannot become supported; missing judges cannot establish a pass; a repair must be reverified; queue ownership must be exclusive; history must restore the selected run; and evidence links must select the correct source.

Validation was performed on 8 October 2026 against the inspected working tree. The backend used its existing Python 3.12 environment and pinned analyzers. No generated program was executed, installed, or used as a dynamic test target.

Frontend component tests ran through Vitest. Browser checks used the production Next.js build, installed Chromium and mocked API/SSE responses.

No paid/live judge requests or live-model benchmark were required for these checks. Provider-adapter failure paths use controlled fixtures/mocks. Therefore the results establish implementation behavior under test conditions, not judge accuracy, actual provider quota, or general model hallucination rates.

5.2 Item Pass/Fail Criteria

The following criteria define observable passing and failing behavior for each verification and workflow area. They guide interpretation of the detailed cases and ensure that missing evidence, infrastructure failure or stale interface state cannot be counted as a successful check.

Item	Passing condition	Failure condition
Intake	Required ambiguity creates questions; valid explicit context is preserved	Missing context silently treated as confirmed; language misidentified
Artifact contract	Exactly one nonempty primary artifact with compatible language and structured metadata	Empty/multiple/truncated artifacts or invalid metadata treated as normal output
Parsing/extraction	Valid representative samples parse and located claims are extracted	Invalid structure missed or claims lose source identity

Item	Passing condition	Failure condition
Symbols/catalogs	Direct conflicts are reported; incomplete context stays uncertain	Partial absence/outage interpreted as proof of nonexistence or support
CoVe/consensus	Contradictions persist; only valid judges count	Confident semantic text overrides an error; missing judges fabricate pass
Policy/repair	Blockers repair/reject; uncertainty warns; repair bounded and reverified	Blocker accepted, retry budget exceeded or unverifiable dependency introduced
Durability	Exclusive job claim, ordered events, preserved partials, guarded cancellation	Duplicate ownership/events, lost evidence or late overwrite of terminal state
Frontend	Selected code/evidence stay aligned; reconnect differs from run status	Stale response changes run or historical selection shifts without user action
Browser layout	Tested widths have no page overflow and source navigation works	Horizontal document overflow, inaccessible evidence or unloaded fonts

A pass is recorded only when its assertion suite ran successfully. A designed test or readable source file alone is not a pass. The isolated backend unit run is included in the later 64-test suite, not counted twice.

5.3 Observed Results

This section summarizes the validation session recorded on 8 October 2026. The table identifies outcomes and evidence artifacts for backend, frontend, browser, build and migration checks; the subsequent discussion explains how the distinct test count and durations should be interpreted.

Validation	Observed outcome	Scope / evidence artifact
Backend full suite	64 passed; 1 deprecation warning; 29.00 s	52 unit cases plus 12 database-backed integration cases; backend-all.xml / backend-all.log
Backend unit-only check	52 passed; 4.43 s	Confirms local analyzer and policy unit behavior; included in full-suite count
Frontend Vitest	23 passed across 5 test files; 3.96 s	API parsing, workspace state and component rendering; observed terminal result
Browser Playwright	8 passed; 14.5 s	Controlled API/SSE scenarios and four viewport widths; browser.log
Frontend lint	Passed; no ESLint warnings or errors	frontend-lint.log
TypeScript check	Passed with no diagnostics	frontend-types.log
Production build	Passed; route generated successfully	frontend-build.log
Database migrations	Upgrade to head completed before integration tests	migrations.log

The distinct automated test count is 95: 64 backend, 23 frontend and eight browser tests. Parametrized parser/artifact inputs are counted as executed cases by their test runners. The recorded duration describes the validation session and must not be presented as application response latency.

5.4 Detailed Backend Test Cases

The following twenty backend cases summarize representative unit and integration checks. Each subsection records its input or preconditions, action, expected behavior, observed outcome and evidence reference, linking parser and policy correctness with durable job execution and persistence.

5.4.1 TC-B01: Language inference and ambiguity

Requirement link: FR-1. Preconditions/data: A short ambiguous task and explicit/inferred language fixtures. Steps: Infer context for ambiguous input and language-specific tasks; inspect clarification flags/questions and assumptions.

Expected result: Blocking ambiguity produces questions; inferred context is marked and explicit language names are retained. Actual result: Pass. Evidence: `test_inference_marks_blocking_ambiguity, backend-all.xml`.

5.4.2 TC-B02: Eight-language parser and extraction matrix

Requirement link: FR-3 / FR-4. Preconditions/data: Representative Python, JavaScript, TypeScript, Java, Go, Rust, C and C++ samples. Steps: Parse valid samples and extract assertions; repeat with malformed source in each adapter.

Expected result: Valid samples produce located claims and available parser coverage; malformed samples produce error evidence. Actual result: Pass. Evidence: `test_all_adapters_parse_and_extract, backend-all.xml`.

5.4.3 TC-B03: Lexical scopes, aliases and invalid APIs

Requirement link: FR-4 / FR-5. Preconditions/data: Python imports, aliases, parameters, shadowing and selected catalog APIs. Steps: Resolve aliases and scopes; test an invented json member and an invalid keyword; inspect linked CoVe checks.

Expected result: Supported lexical bindings remain supported; complete catalog contradictions yield located errors and unsupported checks. Actual result: Pass. Evidence: `test_invented_api_and_invalid_parameters_use_complete_evidence, backend-all.xml`.

5.4.4 TC-B04: Registry absence versus outage

Requirement link: FR-5.3. Preconditions/data: Mocked registry responses and cleared cache. Steps: Return 404 for one package and a connection error for another; compare evidence states.

Expected result: Authoritative 404 is unsupported; network outage is uncertain. A package name alone does not prove nonexistence. Actual result: Pass. Evidence:
`test_registry_absence_and_outage_are_distinct, backend-all.xml`.

5.4.5 TC-B05: Static unsafe-pattern and repetition evidence

Requirement link: FR-4.5 / FR-6.3. Preconditions/data: Local analyzer tools and a source-only unsafe shell sample. Steps: Scan the sample without executing it; compute repetition for duplicated print statements.

Expected result: A local Semgrep rule observation is produced and repetition is nonzero. No generated operation is executed. Actual result: Pass. Evidence:
`test_unsafe_operation_triggers_sast_fallback, backend-all.xml`.

5.4.6 TC-B06: Metric count and missing entropy semantics

Requirement link: FR-6. Preconditions/data: One unsupported and one uncertain claim with unavailable entropy. Steps: Compute metrics and inspect counts, MaHR and null entropy.

Expected result: MiHN is one, MaHR is one, unsupported/uncertain counts are separate, and entropy remains null. Actual result: Pass. Evidence:
`test_metrics_separate_unsupported_uncertain_and_unavailable_entropy, backend-all.xml`.

5.4.7 TC-B07: CoVe contradiction precedence and missing judges

Requirement link: FR-7 / FR-8. Preconditions/data: A claim linked to an error finding and a proposed semantic support response. Steps: Merge the semantic answer; separately request judges with no credentials.

Expected result: Static contradiction remains unsupported; missing providers contribute no valid passing verdict or score. Actual result: Pass. Evidence:
`test_static_contradiction_cannot_be_overridden_by_semantic_check, backend-all.xml`.

5.4.8 TC-B08: Acceptance requires complete evidence

Requirement link: FR-9.1 / FR-9.2. Preconditions/data: Three valid passing judges, supported checks and available parser coverage. Steps: Evaluate policy, then change coverage to partial and evaluate again.

Expected result: Complete configured evidence accepts; changing coverage to partial yields warn, even with passing judges. Actual result: Pass. Evidence:
`test_accept_requires_full_judge_and_analyzer_coverage, backend-all.xml`.

5.4.9 TC-B09: Generation artifact validation

Requirement link: FR-2.4. Preconditions/data: Empty, multiple, malformed or language-conflicting fenced outputs. Steps: Validate controlled invalid artifacts and incorrectly typed/incomplete metadata.

Expected result: Invalid artifacts raise validation errors; required metadata is not silently accepted. Actual result: Pass. Evidence: `test_invalid_artifacts_are_rejected`, `backend-all.xml`.

5.4.10 TC-I01: Exclusive queue claim and persisted evidence links

Requirement link: FR-10.4 / FR-8. Preconditions/data: Migrated isolated PostgreSQL and a queued deterministic task. Steps: Have two owners claim concurrently; execute the claimed run; inspect claims, CoVe foreign keys and ordered events.

Expected result: Exactly one owner claims the job; claim links persist; sequences are contiguous and completion is journaled. Actual result: Pass. Evidence: `test_queue_exclusive_claim_and_claim_evidence_links`, `backend-all.xml`.

5.4.11 TC-I02: Cancellation and expired worker lease

Requirement link: FR-10.4 / reliability. Preconditions/data: Isolated database and leased jobs. Steps: Heartbeat as the owner and a different owner; cancel; then expire a separate lease and sweep it.

Expected result: Only the owner renews; cancellation prevents later execution; expired work becomes interrupted with evidence retained. Actual result: Pass. Evidence: `test_cancellation_and_expired_lease`, `backend-all.xml`.

5.4.12 TC-I03: Clarification resumes the same run

Requirement link: FR-1.5. Preconditions/data: Deterministic request with prompt add. Steps: Execute to clarification, answer with Python numeric requirements, and execute again.

Expected result: The original run changes from `needs_clarification` to `completed` and contains a generated attempt. Actual result: Pass. Evidence: `test_clarification_resumes_same_run`, `backend-all.xml`.

5.4.13 TC-I04: Repair budget and repeated verification

Requirement link: FR-9.3–FR-9.6. Preconditions/data: Deterministic fake-reference request and isolated database. Steps: Run with available repairs, inspect two evaluated attempts, then repeat with zero repairs.

Expected result: First candidate requests repair; repaired candidate warns with lower MiHN in this fixture; zero-repair blocker is rejected. Actual result: Pass. Evidence: `test_repair_limit_reverification_and_hash_stop`, `backend-all.xml`.

5.4.14 TC-I05: HTTP enqueue, replay, cancel and export

Requirement link: FR-10 / API contract. Preconditions/data: TestClient with isolated database dependency overrides. Steps: POST a task, cancel it, replay events after sequence one, export evidence, and submit invalid path/clarification state.

Expected result: Submission is 202 queued; replay excludes prior creation, export is coherent; invalid UUID is 422 and invalid resume is 409. Actual result: Pass. Evidence: `test_http_enqueue_replay_cancel_and_export`, `backend-all.xml`.

5.4.15 TC-I06: Failure retains partial output and stage status

Requirement link: FR-10.3 / reliability. Preconditions/data: Judge method deliberately raises an infrastructure exception. Steps: Execute a valid generation through the failing judge stage and read retained evidence/events.

Expected result: Run is failed; partial code, claims and coverage remain; `judge_pool` has a failed-stage event. Actual result: Pass. Evidence: `test_failure_retains_output_and_marks_stage_failed`, `backend-all.xml`.

5.4.16 TC-I07: Repeated artifact stops repair without acceptance

Requirement link: FR-9.6. Preconditions/data: Repair generator returns the same blocking source as before. Steps: Execute with a larger nominal repair budget and inspect attempts/final reason.

Expected result: Two attempts are retained, run is rejected, and repeated-code reason is recorded rather than exhausting unnecessary retries. Actual result: Pass. Evidence: `test_same_code_hash_stops_repair_without_accepting_blocker`, `backend-all.xml`.

5.4.17 TC-I08: Separate worker entry point

Requirement link: FR-10.4 / deployment. Preconditions/data: Queued run in isolated PostgreSQL; worker `SessionLocal` overridden to the test database. Steps: Claim the job and invoke the worker process function; reopen the stored run/job.

Expected result: Run and job complete through the worker entry point independently of the browser/API submission lifetime. Actual result: Pass. Evidence: `test_separate_worker_process_entrypoint`, `backend-all.xml`.

5.4.18 TC-I09: Incomplete metadata normalization preserves source

Requirement link: FR-2.2 / FR-2.4. Preconditions/data: Valid addition source with incomplete original metadata; controlled normalization response. Steps: Stream the artifact, normalize metadata separately, and inspect final source and provenance.

Expected result: Code is unchanged, `metadata_normalized` is true and the `incomplete-original-metadata` limitation is retained. Actual result: Pass. Evidence: `test_incomplete_metadata_is_normalized_without_changing_code`, `backend-all.xml`.

5.4.19 TC-I10: Repair cannot hide an unverified new dependency

Requirement link: FR-9.4 / FR-5. Preconditions/data: Repair fixture imports an unverified dependency outside supported/allowed evidence. Steps: Run the repair path and inspect findings of the final candidate.

Expected result: The dependency produces `repair.disallowed-dependency` and the remaining blocker is rejected. Actual result: Pass. Evidence: `test_repair_cannot_introduce_unverified_dependency`, `backend-all.xml`.

5.4.20 TC-I11: API pipeline persistence and compatibility stream

Requirement link: FR-2 / FR-10. Preconditions/data: Isolated database and fake generator. Steps: Create a persisted candidate, reopen evidence, and consume compatibility stream events.

Expected result: Attempt/judge evidence is stored; token events appear and the stream ends with `run_completed` evidence. Actual result: Pass. Evidence: `test_pipeline_persists_complete_run`, `backend-all.xml`.

5.5 Frontend and Browser Test Cases

This section presents six frontend and browser case groups covering stream contracts, workspace restoration, historical selection, evidence navigation, policy-state presentation and responsive interaction. Recorded outcomes come from controlled tests and API/SSE fixtures; they do not measure live generator or judge accuracy.

5.5.1 TC-F01: Stream Parsing and Contract Handling

Preconditions: framed stream payloads are supplied through controlled readers. Steps: parse streamed run events, preserve event types and string-based policy outcomes, and verify client interpretation without requiring live network calls. Expected result: JSON framing and policy contracts remain usable by the workspace. Actual result: `frontend/lib/api.test.ts` completed as part of the passing 23-test suite. Scope: parsing behavior; it does not validate external network reliability.

5.5.2 TC-F02: Durable Workspace Restoration and Stale Responses

Preconditions: saved run identifiers and mocked snapshots/events. Steps: restore a prior run, reconnect without changing its execution state, switch to a newer selected run and deliver a stale response from the old one. Expected result: the selected run remains coherent, connection state is separate, and late data cannot overwrite the current selection. Actual result: the `use-workspace DOM` suite passed. Related requirements: FR-10.4 and FR-10.5.

5.5.3 TC-F03: Historical Attempt Selection and New Draft

Steps: select a historical candidate, deliver newer attempt evidence, and create a new draft while a job remains active. Expected result: selected history remains stable and New run does not send a cancellation request. Actual result: the workspace lifecycle assertions passed. This case supports the explicit separation of inspection, draft creation and server cancellation.

5.5.4 TC-F04: Evidence Navigation and Unavailable Data

Steps: render claims/findings, activate linked evidence, navigate to the indicated source line, and inspect null metrics/provider failure results. Expected result: code/evidence remain attempt aligned; missing values display unavailable and failed providers do not display fabricated scores. Actual result: workspace component and EvidencePanel cases passed, including partial-evidence rendering when no complete attempt exists.

5.5.5 TC-B01: Clean, Repaired, Rejected and Degraded Completion

Preconditions: four deterministic API/SSE scenarios. Steps: submit through the implemented composer, verify the visible outcome, inspect Metrics, reload the page, and restore the run. For repaired work, select initial/latest artifacts and check Changes behavior; for degraded work, inspect the provider failure message. Expected result: every fixture outcome remains inspectable after reload and selection stays coherent. Actual result: four Playwright cases passed. These are UI-contract tests, not live judgments of code correctness.

5.5.6 TC-B02: Responsive Layout and Source Navigation

Preconditions: Chromium viewports of 360, 768, 1280 and 1440 pixels. Steps: render the composer; submit with Control+Enter; open findings; activate the source line; assert document width; inspect bundled font loading; emulate reduced-motion preference; capture screens. Expected result: no horizontal page overflow, source highlight present, configured fonts loaded and preference observable. Actual result: all four viewport cases passed. Device coverage is limited to these Chromium scenarios.

5.6 Defect Analysis and Evaluation Limitations

The passing assertions demonstrate that known failure conditions are handled according to the tests. They do not imply that arbitrary generated programs are defect free. In particular, functional correctness is not established by executing the generated artifact; semantic review remains a fallible support signal. A source-only analysis can miss behavior dependent on runtime inputs, environment, third-party implementation or external project files.

Catalog coverage remains deliberately narrow. API absence is only a contradiction where the catalog is complete enough for that conclusion. Tests cover representative aliasing, shadowing and invalid-parameter cases, but the record does not claim complete interprocedural, cross-file or version-aware resolution.

5.7 Risks and Contingencies

The following risk register identifies conditions that can limit verification coverage or disrupt testing. It connects each risk to its impact, a practical mitigation or fallback, and the remaining limitation so that passing controlled tests are not interpreted as universal reliability.

Risk	Testing impact	Mitigation / contingency
Live provider quota or model-access changes	Semantic review cannot be assumed from configured keys	Record unavailable/failed results; use mocks for contract tests; separately check access
Generator nondeterminism	Repeated prompts may produce different claims and failures	Deterministic fixtures for regression; controlled repeated live evaluation as future work
Analyzer absence or timeout	No findings can resemble a clean scan	Explicit coverage state; warn policy; inspect installed tools and logs
Registry outage	Package absence cannot be established	Preserve uncertain status; short-lived cache; do not infer nonexistence
Database/worker unavailability	Runs remain queued or become interrupted	Health diagnosis, separate worker, durable records and lease status
Tests against application data	Integration artifacts could contaminate real history	Isolated migrated PostgreSQL cluster for the definitive test session
Limited browser/device coverage	Untested browsers or assistive technology may differ	Four-width Chromium checks; extend browser and accessibility matrix later
Overinterpretation of scores	Heuristic risk mistaken for correctness probability	Metric help, separate counts/coverage and deterministic policy precedence

5.8 Reproduction and Evidence Location

The companion evidence directory under `reports/final-report/evidence` contains backend JUnit/log output, browser output, migration evidence and frontend check logs. Source test files provide the assertions and fixtures behind each case. The report's tested working tree includes local changes, so reproduction should use the same submitted artifact set rather than assuming the Git revision alone captures every detail.

```
# Backend unit check, from backend
.venv/bin/pytest tests/unit -q

# Full backend checks: use a separate migrated DATABASE_URL
alembic upgrade head
.venv/bin/pytest -q

# Frontend, from frontend
npm test
npm run lint
npx tsc --noEmit
DEHALU_NEXT_DIST_DIR=.next-check npm run build
npm run test:browser
```

The full backend suite requires an isolated PostgreSQL database. The report validation used a temporary cluster; normal users do not need that test-cluster port for ordinary application operation.

6. User Manual

This chapter explains how to install and operate the submitted system. Its seven subsections cover prerequisites, configuration and startup, readiness, main user workflows, history/export, troubleshooting and maintainer validation. Ordinary inspection tasks are separated from contributor checks and live-provider experiments.

6.1 System Requirements and Prerequisites

DeHalu operates as several local processes: the frontend, API, database, background worker and Ollama generator. A modern desktop browser is recommended for detailed evidence inspection, although the composer and navigation support narrow screens. The repository documents Python 3.11+, Node.js 20+, Docker with Compose and an available local Ollama model.

Requirement	Purpose / note
Python 3.11 or newer	Backend service, worker, schema migration and tests

Requirement	Purpose / note
Node.js 20 or newer and npm	Next.js frontend development/build
Docker Compose or compatible PostgreSQL installation	Durable run/evidence storage; documented Compose service uses PostgreSQL 16
Ollama with a selected model already pulled	Local initial generation, structured requests and repairs
Configured judge credentials for full live review	Gemini, Groq and Mistral access; model/quota must be checked separately
Installed backend runtime packages	Tree-sitter grammar pack, Semgrep, SQLAlchemy, FastAPI and provider HTTP access
Modern browser	Composer, streaming code, evidence and history

The project does not provide a claimed one-click installer. Installation uses the source repository and package environments. Deterministic fake mode is useful for demonstrating the workflow without live provider calls, but it is not semantic verification.

6.2 Installation and Configuration

Installation requires the source files, explicit generator/review configuration, a migrated database and separately running application processes. Sections 6.2.1–6.2.4 walk through obtaining the source, selecting configuration, preparing PostgreSQL/backend dependencies, and starting the API, worker and frontend.

6.2.1 Obtain the Source

Use the submitted project folder or the configured source repository address, <https://github.com/FarhanTausif/SPL-3>. Ensure the submission includes backend, frontend, migration and rule files. When using a remote copy, confirm that it includes the local changes described in the report.

```
git clone https://github.com/FarhanTausif/SPL-3.git
cd SPL-3
cp .env.example .env
```

Keep credentials in local environment configuration. Do not place provider keys in prompts, screenshots, report text or committed files.

6.2.2 Select Generator and Review Mode

Set `DATABASE_URL` to the application's database, `OLLAMA_BASE_URL` to the local runtime and `OLLAMA_MODEL` to a model available there. The runbook uses `qwen2.5-coder:1.5b` as a development configuration; code defaults can differ. Model selection should be explicit and checked through the runtime rather than inferred from a default name.

```
DATABASE_URL=postgresql+psycopg://dehalu:dehalu@localhost:5433/dehalu
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=qwen2.5-coder:1.5b
DEHALU_MAX_RETRY=3
DEHALU_PACKAGE_LOOKUPS=false
NEXT_PUBLIC_API_BASE_URL=http://localhost:8000
```

For live judging, configure `GEMINI_API_KEY`, `GROQ_API_KEY` and `MISTRAL_API_KEY`, along with the intended model identifiers. The runbook and Settings contain different defaults in the inspected working tree, so verify the actual chosen configuration.

For a deterministic demonstration, set `DEHALU_ALLOW_FAKE_LLM=true`. Return it to false for live generation/review. Do not interpret this mode as evidence that cloud judges reviewed the artifact.

6.2.3 Start the Database and Install Backend Dependencies

From the repository root, start the documented PostgreSQL service. Port 5433 is the host mapping for container PostgreSQL port 5432. If an existing service already occupies the port, change configuration deliberately rather than terminating unrelated processes.

```
docker compose up -d postgres
docker compose ps postgres
cd backend
python3 -m venv .venv
source .venv/bin/activate
pip install -e ".[test]"
alembic upgrade head
```

Apply migrations before starting normal work. Schema creation is migration-based, and the API is not responsible for creating missing tables automatically.

6.2.4 Start API, Worker and Frontend

Run the API in one terminal and the worker in a second terminal, both with the backend environment activated. A working API with no worker can enqueue work, but its runs may remain queued.

```
# Terminal 1, from backend with environment active
uvicorn dehalu.main:app --host 127.0.0.1 --port 8000 --reload
```

```
# Terminal 2, from backend with environment active
python -m dehalu.worker
```

```
# Terminal 3, from frontend
npm install
npm run dev
```

Ensure Ollama is running and that the configured model is available before a live run. Open <http://localhost:3000> in the browser.

6.3 Readiness and First Use

Open the provider-status control in the application header. Verify database reachability, generator mode/model availability and configured judge status. “Configured” does not mean the provider has successfully reviewed the selected attempt. A “Limited coverage” message means some review evidence may remain unavailable; the user can still inspect what the system produces.

A first task can be explicit and small: “Write a Python function that adds two numbers, using no external dependencies.” This avoids unnecessary ambiguity and makes the expected source easy to inspect. Enter it in Prompt and press Generate or Ctrl/Cmd+Enter.

Wait for the verification decision. If the run completes with warn, read the reason and evidence instead of assuming completed means accepted. In live mode, inspect provider failures or analyzer coverage to understand why unrestricted acceptance was not available.

6.4 Main User Workflows

The seven workflows below follow the user’s path from composing a request to reviewing and copying a candidate. They explain clarification, progress, policy decisions, evidence/source links, metric and judge interpretation, and attempt comparison using the implemented interface controls.

6.4.1 Compose a Task and Supply Context

Write what the code should do, its expected input/output and important constraints. Include the language if known. Use Advanced options for framework, runtime, libraries and maximum repairs. Enter libraries as comma-separated names and constraints on separate lines. The UI preserves these as structured request context.

Examples include a Python grouping utility without external dependencies, a typed TypeScript validation result, or Go JSON decoding with clear errors. A useful prompt specifies the handling of invalid input rather than leaving that behavior implied.

6.4.2 Answer Clarification

If questions appear, read them and supply the missing task context. For a vague “add” prompt, identify the language, inputs and intended operation. Submit the answer through the clarification form. The same run is requeued; its history and identity are preserved.

If the action fails because the run is no longer waiting for clarification, inspect its current state before resubmitting. Do not create multiple submissions only to bypass a state mismatch.

6.4.3 Monitor Generation and Checks

Use the workflow strip to follow intake, generation, static checks, judges, verification and decision. The Code view displays the artifact as tokens arrive, with attempt context visible. Until a policy decision exists, the code is a draft under review.

If the browser connection drops, the server job can continue. The connection message may offer reconnection. Starting New run opens a fresh composer but does not stop the background job; use Cancel when stopping work is intended.

6.4.4 Interpret the Verification Decision

Accept means the current configured policy conditions were satisfied; it does not establish total functional correctness or replace the user's project tests. Warn means no blocking contradiction remains under the completed checks, but some claim, coverage, quality or semantic limitation remains. Reject means a blocking issue was not resolved within the allowed repair path.

Repair is an intermediate decision. Inspect the subsequent artifact and its evidence rather than assuming every rewrite improved the result.

6.4.5 Inspect Claims, Findings and Source Links

Open Evidence and choose Claims to see individual assertions. Filter by supported, unsupported, uncertain or not_checked as relevant. Findings shows analyzer observations by error, warning or info. Search narrows long lists. Expand a row to inspect its message, method, source range and evidence.

Use available evidence links to follow the supporting or contradicting record. Use Line N to open the corresponding candidate's code and highlight that source location.

6.4.6 Interpret Metrics and Judges

MiHN counts unsupported claims. MaHR includes unsupported and uncertain/unreviewed claims; it is not a calibrated failure probability. TR-S measures textual repetition, and Risk combines several heuristic signals. Read tooltips and coverage alongside numeric values. An unavailable entropy value means no suitable measurement was supplied, not that entropy was measured at zero.

Judges shows the provider/model, observed status, verdict/score where valid, rubric material and explanation. Configured credentials can still lead to access or quota failures. Verification shows claim-specific CoVe questions and evidence-linked verdicts; a fluent explanation without support does not establish correctness.

6.4.7 Compare Attempts and Copy Code

Use Inspect to select the desired attempt. The Code panel and evidence use that selection. Changes compares the initial artifact with the selected later artifact; it is disabled for the initial candidate. Review repair summaries and the later policy reason together with the diff.

Copy transfers the displayed source to the clipboard. If clipboard access fails, the interface reports it. Integrate only after examining remaining limitations and running appropriate tests in the user's own controlled project environment.

6.5 History and Evidence Export

The history sidebar supports search, open and refresh. On narrow screens use Open history to reach the drawer. Reload restoration refers to the browser's selected run and the API's stored evidence; it is not an account-based login history.

Export downloads a JSON document containing run context, attempts and available partial evidence. Retain it when sharing findings with a supervisor or reproducing a reported issue. The local prototype does not enforce per-user ownership of history records.

The application does not claim a complete history-deletion or retention-settings flow. Database maintenance remains an operator responsibility.

6.6 Troubleshooting and Recovery

This section maps common operational symptoms to likely causes and supported recovery actions. The table helps distinguish database, worker, model-provider and connection issues; the accompanying guidance explains when configuration changes require process restarts.

Symptom	Likely cause	Supported action
Run stays queued	Worker absent, stopped or using another database	Start/check the worker and confirm shared DATABASE_URL
Database unavailable	Container not ready, wrong port/credentials, missing schema	Check Compose service, connection configuration and Alembic head
Generator unavailable	Ollama not running or model name not installed	Check local runtime and configured OLLAMA_MODEL
Judge unavailable/failed	Missing key, denied model access, quota or provider/schema failure	Inspect provider evidence and correct configuration; retain warn limitation
Evidence reports partial/failed analyzer	Tool absent, unsupported context, parse/tool error or timeout	Confirm installed pinned analyzers and inspect coverage details
Unknown package uncertain	Local catalogs lack evidence or external lookup disabled	Supply context or enable configured metadata lookup if appropriate
Browser cannot reach API	Wrong base URL/CORS origin or server stopped	Verify NEXT_PUBLIC_API_BASE_URL, server address and permitted origin
Stream reconnecting/offline	Network interruption or stream endpoint unavailable	Reconnect/reopen history; server work may still continue
Worker interrupted	Lease expired before completion	Review retained partial evidence and submit a new run to retry
Rejected after repair	Blocker remains, code repeated or retries spent	Inspect failing evidence and improve task/context; do not treat rejected output as verified

After changing environment variables, restart the relevant API/worker processes and rebuild/restart the frontend when public build-time configuration changes. Do not assume a running process has adopted a new file value. When diagnosing provider access, use the repository's readiness tool according to the runbook; a catalog check still does not guarantee generation quota.

6.7 Validation for Maintainers

Maintainers can run backend units first, then integration tests against a disposable migrated database, followed by frontend and browser checks. Generated TypeScript contracts should be regenerated after changing API/domain schemas. This procedure helps keep endpoint payloads, client rendering and persisted evidence aligned.

```
# From repository root, after a schema change
PYTHONPATH=backend/src backend/.venv/bin/python backend/scripts/generate_contracts.py
```

```
# Backend unit tests
cd backend
source .venv/bin/activate
pytest tests/unit
```

```
# Frontend checks, from frontend
npm test
npm run lint
npx tsc --noEmit
```

These contributor operations are separate from ordinary user tasks. A user inspecting a run need not modify source code or regenerate contracts. Live smoke tools can make real provider calls and should be used deliberately with a disposable database; they were not used to claim live semantic accuracy in this report.

7. Conclusion

This chapter assesses the project against its objectives and evidence. Section 7.1 summarizes achieved software outcomes, Section 7.2 states the limits of coverage and evaluation, and Section 7.3 identifies future empirical and technical work. Conclusions remain bounded by the static verification scope and recorded controlled tests.

7.1 Achieved Outcomes

DeHalu delivers an implemented prompt-to-evidence workspace for local code generation with an execution-free verification layer. Its main contribution is a connected audit trail: claims, analyzer findings, coverage, metrics, judges, CoVe, policy and repair attempts remain inspectable together. The workflow preserves uncertainty and prevents a missing check from being treated as a successful review.

The implemented system addresses the objectives of usable task intake, structural/reference checking, transparent policy, bounded mitigation and durable history. The test session recorded 64 passing backend tests, 23 passing frontend tests and eight passing browser tests.

The durable queue and event journal improve operational reliability by separating job execution from the browser connection. Clarification resumes the same run; explicit cancellation protects terminal state; partial evidence survives infrastructure failure; and ineffective repeated repair is stopped.

7.2 Limitations

The system's deterministic scope is strongest for source structure, selected package/API evidence and supported lexical references. Partial catalogs and absent build/project context leave some claims uncertain. Semantic judging remains dependent on external service availability and model judgment. Local generation alone does not make the entire workflow private when cloud judges are enabled.

The report does not establish a benchmark accuracy rate, broad live-model mitigation improvement or guaranteed functional correctness. Generated artifacts were not dynamically executed in the committed pipeline or report-validation session.

7.3 Future Work

Priority future work is a labeled evaluation corpus that separates unsupported claims, uncertainty and genuine semantic defects across languages. Repeated live-model experiments should record provider/model configuration, cost/access limitations and before/after repair behavior without equating simulation with evaluation. Such a study can measure detection and mitigation quality beyond regression-test counts.

Technical extensions include version-aware API catalogs, broader lexical/project context, richer multi-language resolution and empirically calibrated metric interpretation. Operational extensions include authenticated ownership, history retention controls and production deployment hardening.

Any extension should preserve the current evidence rule: direct contradictions remain blocking, missing support remains uncertain, and every repaired candidate is verified again. Within its present boundary, DeHalu provides a practical and explainable way to review generated code before reuse, while keeping the user aware of what was checked and what remains unproven.

References

- [1] Y. Lee et al., “Hallucination by Code Generation LLMs: Taxonomy, Benchmarks, Mitigation, and Challenges,” arXiv preprint arXiv:2504.20799, 2025. Available: <https://arxiv.org/abs/2504.20799>.
- [2] C. Gao et al., “A Systematic Literature Review of Code Hallucinations in LLMs: Characterization, Mitigation Methods, Challenges, and Future Directions for Reliable AI,” arXiv preprint arXiv:2511.00776, 2025. Available: <https://arxiv.org/abs/2511.00776>.
- [3] C. Miranda-Pena, A. Reeson, C. Paris, J. Poon, and J. K. Kummerfeld, “An Empirical Analysis of Static Analysis Methods for Detection and Mitigation of Code Library Hallucinations,” arXiv preprint arXiv:2604.07755v1, 2026. Available: <https://arxiv.org/abs/2604.07755v1>.
- [4] F. Mu et al., “ClarifyGPT: A Framework for Enhancing LLM-Based Code Generation via Requirements Clarification,” *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, Art. 103, 23 pp., July 2024, doi: 10.1145/3660810. Available: <https://doi.org/10.1145/3660810>.
- [5] S. Dhuliawala et al., “Chain-of-Verification Reduces Hallucination in Large Language Models,” *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 3563–3578, 2024, doi: 10.18653/v1/2024.findings-acl.212. Available: <https://aclanthology.org/2024.findings-acl.212/>.
- [6] J. He et al., “LLM-as-a-Judge for Software Engineering: Literature Review, Vision, and the Road Ahead,” *ACM Transactions on Software Engineering and Methodology*, vol. 35, no. 9, pp. 1–30, 2026, doi: 10.1145/3797276. Available: <https://doi.org/10.1145/3797276>.
- [7] Y. Tian et al., “CodeHalu: Investigating Code Hallucinations in LLMs via Execution-based Verification,” arXiv preprint arXiv:2405.00253v4, 2025. Available: <https://arxiv.org/abs/2405.00253v4>.
- [8] B. Yang, M. A. Al Mamun, J. M. Zhang, and G. Uddin, “Hallucination Detection in Large Language Models with Metamorphic Relations,” *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, Art. FSE020, 2025, doi: 10.1145/3715735. Available: <https://doi.org/10.1145/3715735>.
- [9] J. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” *Advances in Neural Information Processing Systems*, vol. 35, 2022. Available: <https://arxiv.org/abs/2201.11903>.
- [10] J. Cheng et al., “Chain-of-Thought Prompting Obscures Hallucination Cues in Large Language Models: An Empirical Evaluation,” *Findings of the Association for Computational Linguistics: EMNLP 2025*, pp. 1272–1305, 2025, doi: 10.18653/v1/2025.findings-emnlp.67. Available: <https://aclanthology.org/2025.findings-emnlp.67/>.
- [11] Abhinaykrishna, “A Comprehensive Guide to Detecting Hallucinations in Large Language Models,” *Medium*, 4 May 2025. [Online]. Available: <https://medium.com/@abhinaykrishna/a-comprehensive-guide-to-detecting-hallucinations-in-large-language-models-9d36c953c62c>. [Accessed: 25 July 2026].

- [12] K. Paul, “LLM Monitoring: Detecting Drift, Hallucinations, and Failures,” Medium, 14 August 2025. [Online]. Available: <https://medium.com/@kuldeep.paul08/llm-monitoring-detecting-drift-hallucinations-and-failures-1028055c1d34>. [Accessed: 25 July 2026].
- [13] A. Biswas and N. Vernier, “Detecting hallucinations with LLM-as-a-judge: Prompt engineering and beyond,” Datadog, 25 August 2025. [Online]. Available: <https://www.datadoghq.com/blog/ai/llm-hallucination-detection/>. [Accessed: 25 July 2026].
- [14] Ollama, “Generate a response,” Ollama API documentation. [Online]. Available: <https://docs.ollama.com/api/generate>. [Accessed: 27 July 2026].
- [15] Tree-sitter, “Introduction,” Tree-sitter documentation. [Online]. Available: <https://tree-sitter.github.io/tree-sitter/>. [Accessed: 4 October 2026].
- [16] Semgrep, “Semgrep documentation.” [Online]. Available: <https://semgrep.dev/docs/>. [Accessed: 4 October 2026].